

OBJECT ORIENTED PROGRAMMING WITH C++

Dr. Puneet Kapoor
Assistant Professor

Shoolini University Centre for Distance and Online Education

Copyright © 2022 Shoolini University Center of Online and Distance Education (SCODE)

All rights reserved. This book or any portion thereof may not be reproduced or used in any manner whatsoever without the express written permission of the publisher except for the use of brief quotations in a book review.

Printed by Shoolini University Center of Online and Distance Education (SCODE)

First printing, 2022.

Shoolini University
Bajhol, Solan, Himachal Pradesh
173229

Editorial Board

Dr. Amar Raj Singh

Director and Professor

Shoolini University Centre for Distance & Online Education

Shoolini University, Solan (HP)

Dr. Pankaj Vaidya

Head of School and Professor

Yogananda School of AI, Computer,
and Data Sciences

Shoolini University, Solan (HP)

Dr. Pooja Verma

Assistant Director and Associate
Professor

Shoolini University Centre for
Distance & Online Education

Dr. Purnima Bali

Associate Professor

Chitrakoot School of Liberal Arts

Shoolini University, Solan (HP)

Dr. Neeraj Gandotra

Professor

Yogananda School of AI, Computer,
and Data Sciences

Shoolini University, Solan (HP)

Shoolini University Centre for Distance and Online Education

Shoolini University

Solan (HP)

Contents

UNIT 1: INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING	1
UNIT 2: DATA TYPES & RULES FOR CODING	10
UNIT 3: CONTROL STATEMENTS	19
UNIT 4: OPERATORS IN C++	32
UNIT 5: LOOP STATEMENTS	49
UNIT 6: ARRAY	64
UNIT 7: FUNCTIONS	78
UNIT 8: POINTERS	87
UNIT 9: OBJECTS AND CLASSES	101
UNIT 10: OPERATOR OVERLOADING	111
UNIT 11: INHERITANCE	120
UNIT 12: VIRTUAL FUNCTIONS	130
UNIT 13: FRIEND FUNCTIONS FRIEND CLASS	141
UNIT 14: C-STRINGS	149

UNIT 1: INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING

CONTENT:

▪ Objectives

1.1 Introduction

1.2 Standard C++ consists of three important parts

1.3 Learning C++

1.4 Use of C++

1.5 Basic Syntax

1.6 Comments in C++

1.7 Summary

1.8 Keywords

1.9 Exercise

1.10 References

OBJECTIVES:

After studying this unit, you will be able to:

- Grasp the fundamental characteristics of C++, including its static typing and support for procedural, object-oriented, and generic programming.
- Comprehend the four pillars of object-oriented development in C++, emphasizing encapsulation, data hiding, inheritance, and polymorphism.
- Recognize the core language, C++ Standard Library, and Standard Template Library, understanding their roles in programming tasks.

- Master the basic syntax of C++ programs, dissect program structures, and learn the steps to compile and execute a C++ program practically.

1.1 INTRODUCTION

C++ is a statically typed, compiled, general-purpose, case-sensitive, free-form programming language that supports procedural, object-oriented, and generic programming. C++ is regarded as a middle-level language, as it comprises a combination of both high-level and low-level language features.

C++ was developed by Bjarne Stroustrup starting in 1979 at Bell Labs in Murray Hill, New Jersey, as an enhancement to the C language and originally named C with Classes but later it was renamed C++ in 1983. C++ is a superset of C, and that virtually any legal C program is a legal C++ program.

Object-Oriented Programming

C++ fully supports object-oriented programming, including the four pillars of object-oriented development:

- Encapsulation
- Data hiding
- Inheritance
- Polymorphism Standard Libraries

1.2 STANDARD C++ CONSISTS OF THREE IMPORTANT PARTS

- The core language giving all the building blocks including variables, data types and literals, etc.
- The C++ Standard Library giving a rich set of functions manipulating files, strings, etc.
- The Standard Template Library (STL) giving a rich set of methods manipulating data structures, etc.

1.3 LEARNING C++

The most important thing while learning C++ is to focus on concepts. The purpose of learning a programming language is to become a better programmer; that is, to become more effective at designing and implementing new systems and at maintaining old ones.

C++ supports a variety of programming styles. You can write in the style of Fortran, C, Smalltalk, etc., in any language. Each style can achieve its aims effectively while maintaining runtime and space efficiency.

1.4 USE OF C++

C++ is used by hundreds of thousands of programmers in essentially every application domain. C++ is being highly used to write device drivers and other software that rely on direct manipulation of hardware under real-time constraints. C++ is widely used for teaching and research because it is clean enough for successful teaching of basic concepts.

1.5 BASIC SYNTAX

When we consider a C++ program, it can be defined as a collection of objects that communicate via invoking each other's methods. Let us now briefly look into what a class, object, methods, and instant variables mean.

- Object - Objects have states and behaviours. Example: A dog has states - colour, name, breed as well as behaviours - wagging, barking, and eating. An object is an instance of a class.
- Class - A class can be defined as a template/blueprint that describes the behaviours/states that object of its type support.
- Methods - A method is basically a behaviour. A class can contain many methods. It is in methods where the logics are written, data is manipulated and all the actions are executed.
- Instant Variables - Each object has its unique set of instant variables. An object's state is created by the values assigned to these instant variables.

1.5.1. C++ Program Structure:

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```



```
cout << "Hello World"; // prints Hello World return 0;

}
```

Let us look at the various parts of the above program:

1. The C++ language defines several headers, which contain information that is either necessary or useful to your program. For this program, the header `<iostream>` is needed.
2. The line using namespace std; tells the compiler to use the std namespace. Namespaces are a relatively recent addition to C++.
3. The next line `// main()` is where program execution begins.' is a single-line comment available in C++. Single-line comments begin with `//` and stop at the end of the line.
4. The line `int main()` is the main function where program execution begins.
5. The next line `cout << "This is my first C++ program."`; causes the message "This is my first C++ program" to be displayed on the screen.
6. The next line `return 0;` terminates `main()` function and causes it to return the value 0 to the calling process.

1.5.2. C++ Keywords

The following list shows the reserved words in C++. These reserved words may not be used as constant or variable or any other identifier names.

asm	else	new	this
auto	enum	operator	throw
bool	explicit	private	true
break	export	protected	try
case	extern	public	typedef
catch	false	register	typeid
char	float	reinterpret_cast	typename
class	for	return	union

const	friend	short	unsigned
const_cast	goto	signed	using
continue	if	sizeof	virtual
default	inline	static	void
delete	int	static_cast	volatile
do	long	struct	wchar_t
double	mutable	switch	while
dynamic_cast	namespace	template	

1.6 COMMENTS IN C++

Program comments are explanatory statements that you can include in the C++ code. These comments help anyone reading the source code. All programming languages allow for some form of comments. C++ supports single-line and multi-line comments. All characters available inside any comment are ignored by C++ compiler. C++ comments start with `/*` and end with `*/`. For example:

```
/* This is a comment */
```

```
/* C++ comments can also
```

```
* span multiple lines
```

```
*/
```

A comment can also start with `//`, extending to the end of the line. For example:

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
cout << "This is my first C++ Program"; // prints Hello World

return 0;

}
```

The braces, {}, indicates a code block; in this case, the C++ code is for the function main. We know that this is a function because of the basic format: first, there is the type of the return value, then the name of the function with a pair of parentheses, which is used to declare the parameters passed to the function (and their types). In this example, the function is called main() and the parentheses are empty, indicating that the function has no parameters. The identifier before the function name (int) says that the function will return an integer. The convention with C++ is that a function called main is the entry point of the executable, that is, when you call the executable from the command line, this will be the first function in your code that will be called. Let's break down the different parts of the program:

1. The C++ language uses "headers" which provide important information for your program. In this case, we need the <iostream> header.
2. The line "using namespace std;" tells the compiler to use the "std" namespace. Think of namespaces as a way to organize code.
3. The line "// main() is where program execution begins." is a comment in C++. Comments are notes in your code for humans to understand. This one starts with // and goes until the end of the line.
4. The line "int main()" is like the central control of the program. It's where everything begins.
5. The next line "cout << "This is my first C++ program";" makes the computer display the message "This is my first C++ program" on the screen.
6. Finally, "return 0;" ends the main() function and sends back the value 0 to finish the program.

1.6.1 Compile and Execute C++ Program:

Let's see how to save, compile, and run the program. Just follow these steps:

Step 1. Open a text editor and write the code as shown above.

Step 2. Save the file with the name "hel.cpp."

Step 3. Open a command prompt and go to the folder where you saved the file.

Step 4. Type 'g++ hel.cpp' and hit enter. It will check your code for errors. If there are no errors, the command prompt will move to the next line, and it will create a file called 'a.out' that you can run.

Step 5. Now, type 'a.out' and press enter to run your program.

Step 6. You'll see 'This is my first C++ program' displayed on the screen.

1.7 SUMMARY

This unit introduces fundamental concepts of the C++ programming language, emphasizing its static typing, procedural, object-oriented, and generic programming support. The objectives focus on building a strong foundation in C++ essentials, ensuring learners understand its distinctive features as a compiled, general-purpose, and case-sensitive language. Object-oriented programming principles, including encapsulation, data hiding, inheritance, and polymorphism, are highlighted as integral aspects of C++ development.

The unit delves into the components of Standard C++, elucidating the core language, C++ Standard Library, and Standard Template Library, elucidating their roles in various programming tasks. Learners are encouraged to explore the versatility of C++ through its support for different programming styles, emphasizing the significance of mastering programming concepts for effective system design and implementation.

A practical aspect of the unit involves dissecting the syntax and structure of a C++ program, using a step-by-step breakdown of a sample program. Additionally, the unit guides learners through the process of saving, compiling, and executing a C++ program, reinforcing practical application. Overall, the objectives aim to equip learners with a comprehensive understanding of C++ fundamentals, object-oriented programming principles, and practical skills for program development and execution.

1.8 KEYWORDS

Compiled, General-purpose, Case-sensitive, Procedural, Object-oriented, Middle-level language, C++ Standard Library, Standard Template Library (STL), Headers, Namespace, Compile and Execute.

1.9 EXERCISE

1. What are the four pillars of object-oriented development supported by C++?
2. Who is the creator of C++, and where was it developed?
3. What is the significance of the C++ Standard Library and the Standard Template Library (STL)?
4. How does C++ support different programming styles?
5. Explain the purpose of the main() function in a C++ program.
6. What is the role of the "using namespace std;" line in a C++ program?
7. How are comments written in C++? Provide examples of both single-line and multi-line comments.
8. Describe the structure of a C++ program using braces and the main() function.
9. What is the purpose of the g++ command in compiling a C++ program?
10. How would you save, compile, and run a C++ program using a text editor and a command prompt?

1.10 REFERENCES

- "The C++ Programming Language" by Bjarne Stroustrup:
 - a) Author: Bjarne Stroustrup (the creator of C++)
 - b) Description: This is considered the definitive book for learning C++. It covers the language in-depth and provides insights into its design and evolution.
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo:
 - a) Authors: Stanley B. Lippman, Josée Lajoie, Barbara E. Moo
 - b) Description: Known for its clear and concise style, this book is suitable for beginners and serves as an excellent reference for more advanced topics.
- "Effective C++: 55 Specific Ways to Improve Your Programs and Designs" by Scott Meyers:
 - a) Author: Scott Meyers

- b) Description: This book provides practical advice and tips for writing better C++ code. It focuses on best practices and common pitfalls.
- "Accelerated C++: Practical Programming by Example" by Andrew Koenig and Barbara E. Moo:
 - a) Authors: Andrew Koenig, Barbara E. Moo
 - b) Description: Aimed at those new to programming, this book takes a practical approach to learning C++. It emphasizes writing real programs early in the learning process.
- "C++ Standard Library - A Tutorial and Reference" by Nicolai M. Josuttis:
 - a) Author: Nicolai M. Josuttis
 - b) Description: This book provides a comprehensive guide to the C++ Standard Library. It covers the containers, algorithms, and other components of the library, offering both tutorials and reference material.

UNIT 2: DATA TYPES & RULES FOR CODING

CONTENT:

- **Objectives**

- 2.1 Rules for writing code**

- 2.2 Variables & Constants**

- 2.3 Data Types**

- 2.4 Variable Types**

- 2.5 Variable Declaration In C++**

- 2.6 Summary**

- 2.7 Keywords**

- 2.8 Exercise**

- 2.9 References**

OBJECTIVES:

After studying this unit, you will be able to:

- Describe the rules for writing code in C++, emphasizing essential practices such as including headers, namespace usage, and proper indentation.
- Explain the importance of following coding rules in C++ for clarity, maintainability, and efficiency.
- List common situations in programming where adherence to coding rules becomes crucial for better code readability and debugging.
- Demonstrate an understanding of key concepts in C++ programming, including variables, constants, data types, and variable types, and their significance in creating effective programs.

2.1 RULES FOR WRITING CODE

Writing a C++ program involves following certain rules and conventions to ensure clarity, maintainability, and efficiency. Here are some essential rules for writing C++ programs:

- **Include Headers:** Always include the necessary headers for the libraries or features you are using. For example, for input/output operations, include `<iostream>`.
- **Namespace Usage:** Use the `using` directive cautiously. Avoid using `using namespace std;` in global scope to prevent naming conflicts. Instead, use it selectively within functions.
- **Comments:** Add comments to explain complex parts of your code or to provide context. Follow a consistent commenting style. Use `//` for single-line comments and `/* */` for multi-line comments.
- **Small case Letters:** C++ is case sensitive language, so always use small case letters while writing instructions.
- **Indentation and Formatting:** Use consistent and readable indentation (usually spaces or tabs). Follow a consistent code formatting style. Tools like `clang-format` or `Artistic Style` can help maintain consistent formatting.
- **Variable Naming:** Choose meaningful and descriptive names for variables. Use `camelCase` or `snake_case` for naming conventions. Avoid single-letter variable names except for loop indices.
- **Constants:** Use `const` for variables that should not be modified. Declare constants using `constexpr`.
- **Avoid Magic Numbers:** Avoid using magic numbers directly in your code. Instead, define constants with meaningful names to represent them.
- **Functions:** Break down your code into functions for better modularity and readability. Follow a consistent naming convention for functions. Declare function prototypes if necessary.
- **Error Handling:** Check for errors and handle them appropriately. Use exception handling for exceptional conditions and return error codes or use other mechanisms for expected errors.
- **Memory Management:** Properly manage memory using `new` and `delete` or, preferably, use smart pointers. Avoid memory leaks and dangling pointers.

- **Avoid Global Variables:** Minimize the use of global variables. They can lead to unexpected side effects and make the code less modular.
- **Avoid GOTO Statements:** Avoid using GOTO statements as they can make the code harder to understand and maintain.
- **Optimization:** Write clear and readable code first. Optimize only when necessary and based on performance profiling.
- **Testing:** Regularly test your code with various inputs. Consider using unit testing frameworks for automated testing.
- **Documentation:** Document your code, especially if it involves complex algorithms or non-trivial logic. Provide information on how to build, run, and test your program.

By following these rules, you can create C++ programs that are easier to read, maintain, and understand, fostering good programming practices.

2.2 VARIABLES & CONSTANTS

In C++, variables and constants are fundamental elements used to store and represent data in a program.

Variables Definition: A variable is a named storage location in memory that holds a value, and this value can be changed during the program execution.

Declaration: To use a variable, you need to declare its type and name. For example:

```
int age;           // Declares an integer variable named 'age'
```

```
double price;      // Declares a double variable named 'price'
```

Initialization: Variables should be initialized before using their values. For example:

```
age = 25;          // Initializes 'age' with the value 25
```

```
price = 19.99;     // Initializes 'price' with the value 19.99
```

Assignment: You can change the value of a variable using the assignment operator (=).

```
age = 30; // Changes the value of 'age' to 30
```

Constants Definition: A constant is a value that cannot be altered during the execution of the program.

Declaration: Constants are typically declared using the const keyword.

```
const int MAX_VALUE = 100; // Declares a constant integer named 'MAX_VALUE' with a value of 100
```

```
const double PI = 3.14; // Declares a constant double named 'PI' with a value of 3.14
```

Initialization: Constants must be initialized at the time of declaration. Once a constant is defined, its value cannot be changed throughout the program.

```
int result = MAX_VALUE * 2; // Using the constant 'MAX_VALUE' in an expression
```

2.2.1 Key Points:

Variables are storage locations that can change during program execution.

- Constants are values that remain fixed throughout the program.
- Variables are declared using a data type, while constants are declared using const with a data type.
- Constants should be initialized at the time of declaration, and their values cannot be modified later.
- Using constants can enhance code readability and maintainability by providing meaningful names to fixed values.

2.3 DATA TYPES

While writing program in any language, you need to use various variables to store various information. Variables are nothing but reserved memory locations to store values. This means

that when you create a variable you reserve some space in memory. You may like to store information of various data types like character, wide character, integer, floating point, double floating point, boolean etc. Based on the data type of a variable, the operating system allocates memory and decides what can be stored in the reserved memory.

The following table shows the variable type, how much memory it takes to store the value in memory, and what is maximum and minimum value which can be stored in such type of variables.

Type	Typical Bit Width	Typical Range
char	1byte	-127 to 127 or 0 to 255
unsigned char	1byte	0 to 255
signed char	1byte	-127 to 127
int	4bytes	-2147483648 to 2147483647
unsigned int	4bytes	0 to 4294967295
signed int	4bytes	-2147483648 to 2147483647
short int	2bytes	-32768 to 32767
unsigned short int	Range	0 to 65,535
signed short int	Range	-32768 to 32767
long int	4bytes	-2,147,483,647 to 2,147,483,647
signed long int	4bytes	same as long int
unsigned long int	4bytes	0 to 4,294,967,295
float	4bytes	+/- 3.4e +/- 38 (~7 digits)
double	8bytes	+/- 1.7e +/- 308 (~15 digits)
long double	8bytes	+/- 1.7e +/- 308 (~15 digits)
wchar_t	2 or 4 bytes	1 wide character

2.4 VARIABLE TYPES

A variable provides us with named storage that our programs can manipulate. Each variable in C++ has a specific type, which determines the size and layout of the variable's memory; the range of values that can be stored within that memory; and the set of operations that can be applied to the variable.

The name of a variable can be composed of letters, digits, and the underscore character. It must begin with either a letter or an underscore. Upper and lowercase letters are distinct because C++ is case-sensitive:

Type	Description
bool	Stores either value true or false.
char	Typically a single octet (one byte). This is an integer type.
int	The most natural size of integer for the machine.
float	A single-precision floating point value.
double	A double-precision floating point value.
void	Represents the absence of type.

2.5 VARIABLE DECLARATION IN C++

A variable declaration provides assurance to the compiler that there is one variable existing with the given type and name so that compiler proceed for further compilation without needing complete detail about the variable. A variable declaration has its meaning at the time of compilation only, compiler needs actual variable declaration at the time of linking of the program.

A variable declaration is useful when you are using multiple files and you define your variable in one of the files which will be available at the time of linking of the program. You will use extern keyword to declare a variable at any place. Though you can declare a variable multiple times in your C++ program, but it can be defined only once in a file, a function or a block of code. Try the following example where a variable has been declared at the top, but it has been defined inside the main function:

```
#include <iostream>

using namespace std;

int main ()

{

int a, b;

int c;

float f;

a = 10;

b = 20;

c = a + b;

cout << c << endl;

f = 70.0/3.0;

cout << f << endl;

return 0;

}
```

When the above code is compiled and executed, it produces the following result:

30

23.3333

2.6 SUMMARY

This unit covers the fundamental aspects of Functional English, emphasizing its importance and applicability in various situations. The objectives include the ability to describe Functional English, explain its significance, and identify scenarios where it is essential. In a parallel context, the unit on "Rules for Writing Code" in C++ outlines crucial practices for program clarity, maintainability, and efficiency. Key rules involve including headers,

cautious use of namespaces, consistent commenting, adherence to naming conventions, and proper memory management. The objectives encompass understanding and applying these rules to create readable and efficient C++ programs. The subsequent sections delve into variables, constants, and data types in C++, highlighting their definitions, declarations, and applications. Finally, the unit addresses variable types, explaining how they determine memory allocation, value ranges, and permissible operations. The content aims to equip learners with practical knowledge and skills in both Functional English and C++ programming, fostering effective communication and coding practices.

2.7 KEY WORDS:

Rules, Coding, Headers, Namespace, Comments, Variables, Constants, Data Types, Declaration

2.8 EXERCISE

1. What are the essential rules for writing C++ programs mentioned in Chapter 2?
2. Why is it important to include headers in a C++ program?
3. How should the using directive be used for namespaces, according to the rules?
4. What is the significance of small case letters in C++ coding rules?
5. Explain the importance of indentation and formatting in C++ code.
6. What are the recommended naming conventions for variables in C++?
7. How are constants declared and initialized in C++?
8. Why is it advised to avoid magic numbers in C++ code?
9. What is the role of functions in maintaining modularity and readability in C++ code?
10. How should error handling be approached in C++ programming according to the rules?
11. What are the memory management practices suggested in C++ programming?
12. Why is it recommended to minimize the use of global variables in C++?
13. Discuss the importance of avoiding GOTO statements in C++ code.
14. When should code optimization be considered in C++ programming?
15. How can testing be effectively performed in C++ programs, as per the rules mentioned?

2.9 REFERENCES

- "The C++ Programming Language" by Bjarne Stroustrup:
 - c) Author: Bjarne Stroustrup (the creator of C++)
 - d) Description: This is considered the definitive book for learning C++. It covers the language in-depth and provides insights into its design and evolution.
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo:
 - c) Authors: Stanley B. Lippman, Josée Lajoie, Barbara E. Moo
 - d) Description: Known for its clear and concise style, this book is suitable for beginners and serves as an excellent reference for more advanced topics.
- "Effective C++: 55 Specific Ways to Improve Your Programs and Designs" by Scott Meyers:
 - c) Author: Scott Meyers
 - d) Description: This book provides practical advice and tips for writing better C++ code. It focuses on best practices and common pitfalls.
- "Accelerated C++: Practical Programming by Example" by Andrew Koenig and Barbara E. Moo:
 - c) Authors: Andrew Koenig, Barbara E. Moo
 - d) Description: Aimed at those new to programming, this book takes a practical approach to learning C++. It emphasizes writing real programs early in the learning process.
- "C++ Standard Library - A Tutorial and Reference" by Nicolai M. Josuttis:
 - c) Author: Nicolai M. Josuttis
 - d) Description: This book provides a comprehensive guide to the C++ Standard Library. It covers the containers, algorithms, and other components of the library, offering both tutorials and reference material.

UNIT 3: CONTROL STATEMENTS

CONTENT:

▪ Objectives

3.1 Introduction

3.2 Conditional Statement

3.3 if Statement

3.4 if-else Statement

3.5 else-if Statement

3.6 Nested if-else

3.7 Switch Case Statement

3.8 Summary

3.9 Keywords

3.10 Exercise

3.11 References

OBJECTIVES:

After studying this unit, you will be able to:

- Describe control statements in C++
- Explain the purpose of control statements in programming
- List and understand different types of control statements in C++
- Identify the structure and syntax of conditional statements in C++

3.1 INTRODUCTION

Control statements in C++ are used to control the flow of a program. They determine the order in which the statements are executed based on certain conditions. C++ provides several types of control statements, including:

3.2 CONDITIONAL STATEMENT

The "if" statement in C++ is used for conditional execution of code. It allows you to execute a block of code if a certain condition is true. If the condition is false, the code inside the "if" block is skipped. Here's the basic structure of an "if" statement:

3.3 IF STATEMENT

It is used to execute a block of code if a given condition is true.

```
if (condition) {  
  
    // Code to be executed if the condition is true  
  
}
```

In Following example, we first declare an integer variable number and then prompt the user to enter a number. We use an "if" statement to check whether the entered number is positive. If the condition `number > 0` is true, the code inside the "if" block is executed, and it prints "The number is positive." If the condition is false, the code inside the "if" block is skipped, and the program proceeds to the next line after the "if" statement. In either case, the program will print "End of program."

So, if you enter a positive number when running this program, you will see "The number is positive" in the output. If you enter a non-positive number (zero or negative), that message will not be displayed.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
int number;

// Prompt the user to enter a number

cout << "Enter a number: ";

cin >> number;

// Check if the number is positive

if (number > 0) {

    cout << "The number is positive." << endl;

}

cout << "End of program." << endl;

return 0;

}
```

3.4 IF-ELSE STATEMENT

It is used to execute one block of code if a condition is true and another block if the condition is false.

```
if (condition) {

    // Code to be executed if the condition is true

} else {

    // Code to be executed if the condition is false

}
```

In Following example, after the user enters a number, the program uses an "if-else" statement to check whether the number is positive. If the condition `number > 0` is true, it prints "The number is positive." If the condition is false (i.e., the number is not greater than zero), it

executes the code within the "else" block and prints "The number is not positive (zero or negative)." Finally, the program prints "End of program" regardless of the condition's outcome.

```
#include <iostream>

using namespace std;

int main() {

    int number;

    // Prompt the user to enter a number

    cout << "Enter a number: ";

    cin >> number;

    // Check if the number is positive or not

    if (number > 0) {

        cout << "The number is positive." << endl;

    } else {

        cout << "The number is not positive (zero or negative)." << endl;

    }

    cout << "End of program." << endl;

    return 0;

}
```

3.5 ELSE-IF STATEMENT

It is used to test multiple conditions one by one and execute the block of code associated with the first true condition.

```
if (condition1) {
```

```
// Code to be executed if condition1 is true

} else if (condition2) {

    // Code to be executed if condition2 is true

} else {

    // Code to be executed if none of the conditions are true

}
```

In this example, we'll check if a number is positive, negative, or zero and display an appropriate message based on the condition:

```
#include <iostream>

using namespace std;

int main() {

    int number;

    // Prompt the user to enter a number

    cout << "Enter a number: ";

    cin >> number;

    // Check if the number is positive, negative, or zero

    if (number > 0) {

        cout << "The number is positive." << endl;

    } else if (number < 0) {

        cout << "The number is negative." << endl;

    } else {
```

```
        cout << "The number is zero." << endl;

    }

    cout << "End of program." << endl;

    return 0;

}
```

3.6 NESTED IF-ELSE

In C++, a nested if statement is an if statement that is placed inside another if statement. This allows you to create a more complex decision-making structure by evaluating multiple conditions.

Here's the basic syntax of a nested if statement in C++:

```
if (condition1) {

    // Code to be executed if condition1 is true

    if (condition2) {

        // Code to be executed if both condition1 and condition2 are true

    }

    else {

        // Code to be executed if condition1 is true but condition2 is false

    }

}

else {

    // Code to be executed if condition1 is false

}
```

In this example, there are two levels of conditions. The inner if statement is nested inside the outer if statement. The code inside the inner if statement is only executed if both condition1 and condition2 are true. Here's a simple real-world example to illustrate the concept of nested if statements:

```
#include <iostream>

using namespace std;

int main() {

    int age;

    char gender;

    cout << "Enter your age: ";

    cin >> age;

    if (age >= 18) {

        cout << "Are you male or female? (M/F): ";

        cin >> gender;

        if (gender == 'M') {

            cout << "You are an adult male.";

        }

        else if (gender == 'F') {

            cout << "You are an adult female.";

        }

        else {

            cout << "Invalid gender input.";
```

```
    }  
  
}  
  
else {  
  
    cout << "You are a minor."  
  
}  
  
return 0;  
  
}
```

In this example, the program first checks if the person is 18 or older. If yes, it then asks for the gender and provides different messages based on the age and gender. The nested if statement helps in creating a more detailed decision-making process.

3.7 SWITCH CASE STATEMENT

In C++, the switch statement is a control statement that is used to select one block of code to execute from several options, based on the value of an expression. It provides a way to make multi-way decisions by evaluating the expression and comparing it to various constant values, known as "case labels." When a match is found, the corresponding block of code is executed, and the switch statement is exited.

The basic structure of a switch statement looks like this:

```
switch (expression) {  
  
    case value1:  
  
        // Code to be executed if expression equals value1  
  
        break;  
  
    case value2:  
  
        // Code to be executed if expression equals value2
```

```
        break;

// ...

default:

    // Code to be executed if none of the cases match

}
```

Here's a breakdown of the key elements in a switch statement:

- **expression:** The expression whose value will be evaluated and compared with the case labels.
- **case value1:** This is a case label that specifies a constant value to compare with the expression. If the expression matches value1, the code block associated with it will be executed.
- **break:** The break statement is used to exit the switch statement after a case is executed. Without a break, the program would continue to execute the code in subsequent cases until a break is encountered or the switch statement is finished.
- **default:** The default case is optional and is executed when none of the case labels match the expression. It is like the "else" case in an "if-else" statement.

Here's a simple example of a switch statement:

```
#include <iostream>

using namespace std;

int main() {

    int choice;

    cout << "Choose an option (1, 2, or 3): ";

    cin >> choice;

    switch (choice) {
```


case 1:

```
cout << "You selected option 1." << endl;
```

```
break;
```

case 2:

```
cout << "You selected option 2." << endl;
```

```
break;
```

case 3:

```
cout << "You selected option 3." << endl;
```

```
break;
```

default:

```
cout << "Invalid choice." << endl;
```

```
}
```

```
return 0;
```

```
}
```

In this example, the user is prompted to choose an option, and the switch statement evaluates the choice variable to determine which option was selected. The code block associated with the matching case label is executed, and a message is displayed. If none of the case labels match, the default case is executed, displaying "Invalid choice."

3.8 SUMMARY

The presented content focuses on control statements in C++, particularly delving into the concepts of conditional statements like "if," "if-else," "else-if," and other related structures. In C++ programming, control statements play a crucial role in directing the flow of a program based on specified conditions. The "if" statement allows for the execution of a block of code if a given condition is true, while the "if-else" statement extends this capability by enabling

the execution of different blocks for true and false conditions. The "else-if" statement proves valuable when handling multiple conditions, executing the block associated with the first true condition. Additionally, the content explores the nested "if-else" construct, illustrating how it creates a more intricate decision-making structure by evaluating multiple conditions.

Furthermore, the material introduces the switch case statement, a powerful control statement in C++ used for multi-way decision-making. It allows the selection of one block of code from several options based on the value of an expression. The summary encapsulates the objectives of each section, emphasizing the importance of understanding and effectively implementing these control statements to enhance programming capabilities and decision logic in C++.

3.9 KEYWORDS

Control statements, Conditional statement, if statement ,if-else statement, else-if statement, Nested, if-else, Switch case statement, break, default.

3.10 EXERCISE

1. What is the purpose of control statements in C++?
2. Explain the basic structure of an "if" statement in C++.
3. How does the "if-else" statement work in C++? Provide an example.
4. When is the "else-if" statement used, and how does it differ from the "if-else" statement? Show an example.
5. What is a nested if-else statement? Provide a syntax example and a real-world scenario where it might be used.
6. Describe the basic structure of a switch case statement in C++.
7. What is the purpose of the "break" statement in a switch case statement?
8. When does the code inside the "default" case of a switch statement get executed?
9. Provide an example of using a switch case statement in a C++ program.
10. How does the switch case statement compare to a series of if-else statements in terms of readability and execution?
11. What does the "if" statement in C++ do?
 - a. Prints output to the console
 - b. Executes a block of code based on a condition
 - c. Defines constants
 - d. Performs arithmetic operations

12. When using the "if-else" statement, what happens if the condition in the "if" part is false?
- a. The program terminates.
 - b. The code inside the "if" block is skipped, and the code in the "else" block is executed.
 - c. The code inside the "else" block is skipped.
 - d. An error is generated.
13. What is the purpose of the "break" statement in a switch case statement?
- a. To start the switch statement
 - b. To end the program
 - c. To exit the switch statement after a case is executed
 - d. To skip the current case and move to the next one
14. In a switch case statement, what is the "default" case used for?
- a. To specify the default value of a variable
 - b. To define constants
 - c. To execute a block of code when none of the cases match the expression
 - d. To break out of the switch statement
15. What does a nested if-else statement in C++ allow you to do?
- a. Perform complex arithmetic operations
 - b. Create a more detailed decision-making process by evaluating multiple conditions
 - c. Define constants within the if statement
 - d. Terminate the program
16. Which control statement is used to select one block of code to execute from several options based on the value of an expression?
- a. if statement
 - b. switch case statement
 - c. else-if statement
 - d. for loop
17. What happens if the "break" statement is omitted in a switch case statement?
- a. It triggers an error.
 - b. The program continues to execute the code in subsequent cases.
 - c. The switch statement is ignored.
 - d. The program terminates.

18. Which statement is used to execute one block of code if a condition is true and another block if the condition is false?
- if statement
 - else-if statement
 - switch case statement
 - if-else statement

3.11 REFERENCES

- "The C++ Programming Language" by Bjarne Stroustrup:
 - Author: Bjarne Stroustrup (the creator of C++)
 - Description: This is considered the definitive book for learning C++. It covers the language in-depth and provides insights into its design and evolution.
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo:
 - Authors: Stanley B. Lippman, Josée Lajoie, Barbara E. Moo
 - Description: Known for its clear and concise style, this book is suitable for beginners and serves as an excellent reference for more advanced topics.
- "Effective C++: 55 Specific Ways to Improve Your Programs and Designs" by Scott Meyers:
 - Author: Scott Meyers
 - Description: This book provides practical advice and tips for writing better C++ code. It focuses on best practices and common pitfalls.
- "Accelerated C++: Practical Programming by Example" by Andrew Koenig and Barbara E. Moo:
 - Authors: Andrew Koenig, Barbara E. Moo
 - Description: Aimed at those new to programming, this book takes a practical approach to learning C++. It emphasizes writing real programs early in the learning process.
- "C++ Standard Library - A Tutorial and Reference" by Nicolai M. Josuttis:
 - Author: Nicolai M. Josuttis
 - Description: This book provides a comprehensive guide to the C++ Standard Library. It covers the containers, algorithms, and other components of the library, offering both tutorials and reference material.

UNIT 4: OPERATORS IN C++

CONTENT:

▪ Objectives

4.1 Introduction

4.2 Arithmetic Operators

4.3 Relational Operator

4.4 Logical Operator

4.5 Bitwise Operator

4.6 Assignment Operator

4.7 Operator Precedence in C++

4.8 Increment and Decrement Operator

4.9 Conditional operators

4.10 Summary

4.11 Keywords

4.12 Exercise

4.13 References

OBJECTIVES:

After studying this unit, you will be able to:

- Describe and understand various types, including arithmetic, relational, logical, bitwise, and assignment operators.
- Explain usage of addition, subtraction, multiplication, division, and modulus operators.
- Explore and provide examples for equality, inequality, greater than, and less than comparisons.
- Understand and illustrate application of logical (AND, OR, NOT) and bitwise operators in C++.

4.1 INTRODUCTION

An operator is like a special symbol that tells the computer to do certain math or logic tasks. C++ has lots of these built-in operators. There are different types of operators:

1. Arithmetic Operators: These are for math, like addition and subtraction.
2. Relational Operators: They help compare values, like checking if one number is greater than another.
3. Logical Operators: Used for making logical decisions, such as "and" and "or."
4. Bitwise Operators: Work on individual bits in binary numbers.
5. Assignment Operators: Set values to variables.
6. Misc Operators: Others that don't fit in the above categories.

In this chapter, we'll look at each type of operator one by one and see how they work in C++.

4.2 ARITHMETIC OPERATORS

Let us assume that a holds the value 10 and b holds the value 15

Operator	Description	Example
+	Adds two operands	A + B will give 25
-	Subtracts second operand from the first	A - B will give -5
*	Multiplies both operands	A * B will give 150
/	Divides numerator by denominator	B / A will give 1.5
%	Modulus Operator and remainder of after an integer division	B % A will give 5

Let us understand the concept of arithmetic operators through the basic code:

```
#include <iostream>
```

```
Using namespace std;
```

```
int main() {
```

```
    int a = 10;
```

```
    int b = 5;
```

```
    // Addition
```

```
    int sum = a + b;
```

```
    cout << "Sum: " << sum << std::endl;
```

```
    // Subtraction
```

```
    int difference = a - b;
```

```
    cout << "Difference: " << difference << std::endl; // Multiplication
```

```
    int product = a * b;
```

```
    std::cout << "Product: " << product << std::endl;
```

```
    // Division
```

```
    int quotient = a / b;
```

```
    std::cout << "Quotient: " << quotient << std::endl;
```

```
    // Modulus (remainder after division)
```

```
    int remainder = a % b;
```

```
    std::cout << "Remainder: " << remainder << std::endl;
```

```
    return 0; }
```

4.3 RELATIONAL OPERATOR

Operator	Description	Example
==	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(A == B) is not true.
!=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>=	Checks if the value of left	(A >= B) is not true.

Let us understand the concept of relational operators through the simple code:

```
#include <iostream>
```

```
Using namespace std;
```

```
int main() {
```

```
    int x = 5;
```

```
    int y = 10;
```

```
// Equal to
```

```
    bool isEqual = (x == y);
```



```
cout << "x is equal to y: " << isEqual << endl; // Not equal to
```

```
bool isNotEqual = (x != y);
```

```
std::cout << "x is not equal to y: " << isNotEqual << std::endl;
```

```
// Greater than
```

```
bool isGreater = (x > y);
```

```
std::cout << "x is greater than y: " << isGreater << std::endl;
```

```
// Less than
```

```
bool isLess = (x < y);
```

```
std::cout << "x is less than y: " << isLess << std::endl;
```

```
// Greater than or equal to
```

```
bool isGreaterOrEqual = (x >= y);
```

```
std::cout << "x is greater than or equal to y: " << isGreaterOrEqual << std::endl;
```

```
// Less than or equal to
```

```
bool isLessOrEqual = (x <= y);
```

```
std::cout << "x is less than or equal to y: " << isLessOrEqual << std::endl;
```

```
return 0;
```

```
}
```

4.4 LOGICAL OPERATOR

Let us assume that a holds the value 1 and b holds the value 0:

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then condition becomes true.	(A && B) is false.
 	Called Logical OR Operator. If any of the two operands is non- zero, then condition becomes true.	(A B) is true.
!	Called Logical NOT Operator. Use to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make false.	!(A && B) is true.

Here is an example for Logical Operators:

```
#include <iostream>

using namespace std;

int main() {

    // Logical AND (&&) example

    bool condition1 = true;

    bool condition2 = false;

    if (condition1 && condition2) {

        cout << "Both conditions are true." << endl;

    } else {

        cout << "At least one of the conditions is false." << endl;
```

```
}

// Logical OR (||) example

if (condition1 || condition2) {

    cout << "At least one of the conditions is true." << endl;

} else {

    cout << "Both conditions are false." << endl;

}

// Logical NOT (!) example

if (!condition1) {

    cout << "Condition1 is false." << endl;

} else {

    cout << "Condition1 is true." << endl;

}

return 0;

}
```

In this example:

- The && operator checks if both condition1 and condition2 are true.
- The || operator checks if at least one of condition1 or condition2 is true.
- The ! operator negates the value of condition1.

Remember that the result of a logical operation is a Boolean value (true or false). These operators are commonly used in conditional statements and loops to control the flow of a program based on certain conditions.

4.5 BITWISE OPERATOR

Bitwise operators are like little workers that do tasks on individual bits (0s and 1s) in binary numbers. They work by comparing or combining bits one at a time. Here's how they behave:

p	q	p & q	p q	p ^ q
0	0	0	0	0
0	1	0	1	1
1	1	1	1	0
1	0	0	1	1

Let us understand through the simple snippet of the code:

4.5.1 Bitwise AND (&):

Performs a bitwise AND operation between each pair of corresponding bits. If both bits are 1, the result is 1; otherwise, it's 0.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int a = 5; // Binary: 0101
```

```
    int b = 3; // Binary: 0011
```

```
    int result = a & b; // Binary: 0001 (1 in decimal)
```

```
    cout << "Bitwise AND: " << result << endl;
```

```
    return 0; }
```

4.5.2 Bitwise OR (|):

Performs a bitwise OR operation between each pair of corresponding bits. If at least one of the bits is 1, the result is 1; otherwise, it's 0.

```
#include <iostream>

using namespace std;

int main() {

    int a = 5; // Binary: 0101

    int b = 3; // Binary: 0011

    int result = a | b; // Binary: 0111 (7 in decimal)

    cout << "Bitwise OR: " << result << endl;

    return 0;

}
```

4.5.3 Bitwise XOR (^):

Performs a bitwise exclusive OR operation between each pair of corresponding bits. If the bits are different, the result is 1; if they are the same, the result is 0.

```
#include <iostream>

using namespace std;

int main() {

    int a = 5; // Binary: 0101

    int b = 3; // Binary: 0011

    int result = a ^ b; // Binary: 0110 (6 in decimal)
```

```
    cout << "Bitwise XOR: " << result << endl;

    return 0;

}
```

4.5.4 Left Shift (<<) and Right Shift (>>):

Shifts the bits of the left operand to the left or right by the number of positions specified by the right operand. Filled positions are usually replaced by zeros.

```
#include <iostream>

using namespace std;

int main() {

    int a = 5; // Binary: 0101

    int resultLeftShift = a << 1; // Binary: 1010 (10 in decimal)

    int resultRightShift = a >> 1; // Binary: 0010 (2 in decimal)

    cout << "Left Shift: " << resultLeftShift << endl;

    cout << "Right Shift: " << resultRightShift << endl;

    return 0;

}
```

These bitwise operators are often used in low-level programming, such as working with device registers, graphics programming, or other situations where manipulation at the bit level is necessary.

4.6 ASSIGNMENT OPERATOR

Operator	Description	Example
=	Simple assignment operator, Assigns values from right side operands to left side operand.	$C = A + B$ will assign value of $A + B$ into C
+=	Add AND assignment operator, It adds right operand to the left operand and assign the result to left operand.	$C += A$ is equivalent to $C = C + A$
-=	Subtract AND assignment operator, It subtracts right operand from the left operand and assign the result to left operand.	$C -= A$ is equivalent to $C = C - A$
*=	Multiply AND assignment operator, It multiplies right operand with the left operand and assign the result to left operand.	$C *= A$ is equivalent to $C = C * A$
/=	Divide AND assignment operator, It divides left operand with the right operand and assign the result to left operand.	$C /= A$ is equivalent to $C = C / A$
%=	Modulus AND assignment operator, It takes modulus using two operands and assign the result to left operand.	$C \% = A$ is equivalent to $C = C \% A$
<<=	Left shift AND assignment operator	$C <<= 2$ is same as $C = C << 2$
>>=	Right shift AND assignment operator	$C >>= 2$ is same as $C = C >> 2$
&=	Bitwise AND assignment operator.	$C \&= 2$ is same as $C = C \& 2$
^=	Bitwise exclusive OR and assignment operator.	$C \wedge= 2$ is same as $C = C \wedge 2$
=	Bitwise inclusive OR and assignment operator.	$C = 2$ is same as $C = C 2$

4.7 OPERATOR PRECEDENCE IN C++

Operator precedence is like a set of rules that decide which parts of a math expression to calculate first. Some operators, like multiplication, have higher precedence, meaning they come first before lower-precedence ones, like addition. For instance, in the expression $x = 7 + 3 * 2$, the multiplication ($3 * 2$) happens first because it has higher precedence than addition. So, the result is $7 + 6$, which equals 13.

In simple terms, higher precedence operators go first when you're solving an expression. They are at the top of the precedence table, and lower-precedence ones are at the bottom.

Category	Operators	Associativity
Postfix	() [] -> .	Left to right
Unary	+ - ! ~ ++ --	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	? :	Right to left
Assignment	= += -= *= /= %= &= ^= = <<= >>=	Right to left
Comma	,	Left to right

4.8 INCREMENT AND DECREMENT OPERATOR

In C++, the increment and decrement operators are used to increase or decrease the value of a variable by 1. These operators are shorthand notations and are commonly employed in loops and other situations where you need to perform repetitive arithmetic operations.

The increment operator (++) adds 1 to the value of the variable, while the decrement operator (--) subtracts 1 from the value. Here's how they work:

4.8.1 Increment Operator (++):

- **Postfix Increment (i++):** It first uses the current value of the variable and then increments it.

```
int i = 5;
```

```
int result = i++; // result is assigned 5, then i becomes 6
```

- **Prefix Increment (++i):** It increments the value of the variable and then uses the updated value.

```
int i = 5;
```

```
int result = ++i; // i becomes 6, then result is assigned 6
```

4.8.2 Decrement Operator (--):

- **Postfix Decrement (i--):** It first uses the current value of the variable and then decrements it.

```
int i = 5;
```

```
int result = i--; // result is assigned 5, then i becomes 4
```

- **Prefix Decrement (--i):** It decrements the value of the variable and then uses the updated value.

```
int i = 5;
```

```
int result = --i; // i becomes 4, then result is assigned 4
```

It's important to note that if the increment or decrement operators are used within a larger expression, the order in which the value is updated may affect the overall result.

```
int x = 5;
```

```
int y = 2 * ++x; // y is assigned 12 (2 * (5 + 1))
```

In the example above, the value of x is first incremented, and then the result is used in the multiplication operation.

4.9 CONDITIONAL OPERATORS:

In C++, the conditional operator, often referred to as the ternary operator, provides a concise way to express conditional statements. The syntax for the conditional operator is:

```
condition ? expression_if_true : expression_if_false;
```

Here, condition is a Boolean expression. If the condition is true, the operator evaluates to expression_if_true; otherwise, it evaluates to expression_if_false. The result of the conditional operator is the value of the expression that gets chosen.

Here's a simple example:

```
#include <iostream>

using namespace std;

int main() {

    int x = 5;

    int y = 10;

    // If x is greater than y, assign x to result; otherwise, assign y

    int result = (x > y) ? x : y;

    cout << "The larger value is: " << result << endl;

    return 0;

}
```

In this example, if x is greater than y, the value of x is assigned to result; otherwise, the value of y is assigned to result.

The conditional operator is a concise alternative to the if-else statement in cases where you want to assign a value based on a condition. However, it's important to use it judiciously, as excessive use in complex expressions might reduce code readability.

4.10 SUMMARY

In this unit, the focus is on comprehending operators in C++, encompassing arithmetic, relational, logical, bitwise, and assignment operators. The first objective aims to provide a broad understanding of these fundamental elements in programming. The second objective delves into the specifics of arithmetic operators, elucidating their purpose and demonstrating practical usage through examples. Moving forward, the third objective emphasizes relational operators, illustrating their role in comparisons like equality, inequality, and relative magnitude. Lastly, the fourth objective combines the exploration of logical and bitwise operators, elucidating their functions and demonstrating their application in C++.

This unit seeks to equip learners with a concise yet comprehensive grasp of operators, facilitating their ability to construct meaningful and efficient code. The objectives, presented in a succinct manner, guide learners through fundamental concepts, practical implementations, and the essential distinctions between various types of operators. The unit's structure ensures a systematic and accessible approach to learning these crucial aspects of C++ programming, laying the groundwork for proficient and effective coding practices.

4.11 KEY WORDS

Operators, arithmetic, Logical, Relational, Bitwise, Precedence, Modulo Division

4.12 EXERCISE

1. What is the difference between the "++i" and "i++" increment operators in C++?
2. Explain the ternary operator (? :) in C++ and provide an example of its usage.
3. How does the sizeof operator work in C++? Give an example.
4. What is the purpose of the comma operator in C++? Provide an example where it can be useful.
5. Explain the concept of operator overloading in C++. Provide an example of overloading a binary operator.
6. Differentiate between the logical AND (&&) and bitwise AND (&) operators in C++.

7. What is the significance of the `const` keyword in the context of C++ operators?
8. Explain the concept of short-circuit evaluation in C++. How is it related to logical operators?
9. Describe the difference between the prefix and postfix forms of the increment and decrement operators (`++` and `--`) in C++.
10. How does the conditional operator (`?:`) work in C++? Provide an example of its usage in a statement.
11. Which of the following is the correct way to increment a variable `x` by 1 in C++?
 - a) `x = x + 1;`
 - b) `x += 1;`
 - c) `x++;`
 - d) All of the above
12. What does the bitwise XOR operator (`^`) do in C++?
 - a) Performs logical OR operation
 - b) Performs logical AND operation
 - c) Performs bitwise XOR operation
 - d) Performs bitwise NOT operation
13. What is the purpose of the `sizeof` operator in C++?
 - a) Returns the size of a variable or data type
 - b) Returns the address of a variable
 - c) Returns the value of a variable
 - d) Returns the type of a variable
14. Which operator is used to access the value at a specific memory address in C++?
 - a) `*` (dereference operator)
 - b) `&` (address-of operator)
 - c) `cc ->` (member access operator)
 - d) `[]` (subscript operator)
15. In C++, what is the purpose of the conditional operator (`?:`)?
 - a) Bitwise conditional operation
 - b) Ternary conditional operation
 - c) Logical conditional operation
 - d) Arithmetic conditional operation

Answers:

11. c) x++;
12. c) Performs bitwise XOR operation
13. a) Returns the size of a variable or data type
14. a) * (dereference operator)
15. b) Ternary conditional operation

4.13 REFERENCES

- "The C++ Programming Language" by Bjarne Stroustrup:
 - Author: Bjarne Stroustrup (the creator of C++)
 - Description: This is considered the definitive book for learning C++. It covers the language in-depth and provides insights into its design and evolution.
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo:
 - Authors: Stanley B. Lippman, Josée Lajoie, Barbara E. Moo
 - Description: Known for its clear and concise style, this book is suitable for beginners and serves as an excellent reference for more advanced topics.
- "Effective C++: 55 Specific Ways to Improve Your Programs and Designs" by Scott Meyers:
 - Author: Scott Meyers
 - Description: This book provides practical advice and tips for writing better C++ code. It focuses on best practices and common pitfalls.
- "Accelerated C++: Practical Programming by Example" by Andrew Koenig and Barbara E. Moo:
 - Authors: Andrew Koenig, Barbara E. Moo
 - Description: Aimed at those new to programming, this book takes a practical approach to learning C++. It emphasizes writing real programs early in the learning process.
- "C++ Standard Library - A Tutorial and Reference" by Nicolai M. Josuttis:
 - Author: Nicolai M. Josuttis
 - Description: This book provides a comprehensive guide to the C++ Standard Library. It covers the containers, algorithms, and other components of the library, offering both tutorials and reference material.

UNIT 5: LOOP STATEMENTS

CONTENT:

- **Objectives**
 - 5.1 Introduction**
 - 5.2 For Loop**
 - 5.3 While Loop**
 - 5.4 Do-while Loop**
 - 5.5 Loop Control Statements**
 - 5.6 The Infinite Loop**
 - 5.7 Summary**
 - 5.8 Keywords**
 - 5.9 Exercise**
 - 5.10 References**

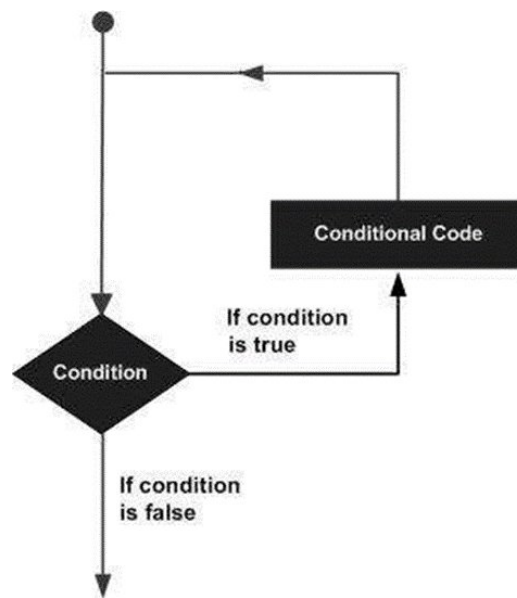
OBJECTIVES:

After studying this unit, you will be able to:

- Grasp the concept of loops in C++ for repetition.
- Identify the three main types: for, while, and do-while.
- Understand the structure of for loops and their application.
- Gain practical knowledge through a simple example.

5.1 INTRODUCTION

Sometimes, you want to do the same thing multiple times in a program. Normally, code runs from the top to the bottom, one line after another. But to make your program more flexible, you can use something called a "loop." A loop lets you repeat a statement or a group of statements over and over again. This is how it generally looks in most programming languages:



Loops in C++ are control structures that allow you to repeatedly execute a block of code if a certain condition is met. They are used for tasks that require repetitive execution, such as iterating over elements in an array, performing calculations, or reading data from a file. C++ provides three main types of loops:

for Loop:

The for loop is used when you know the number of iterations you want to perform in advance. It consists of three parts: initialization, condition, and increment/decrement. The loop continues to execute if the condition is true.

while Loop:

The while loop is used when you want to repeatedly execute a block of code if a condition is true. The condition is checked at the beginning of each iteration, and if it's true, the loop continues.

do-while Loop:

The do-while loop is like the while loop, but the condition is checked at the end of each iteration, which ensures that the loop body is executed at least once.

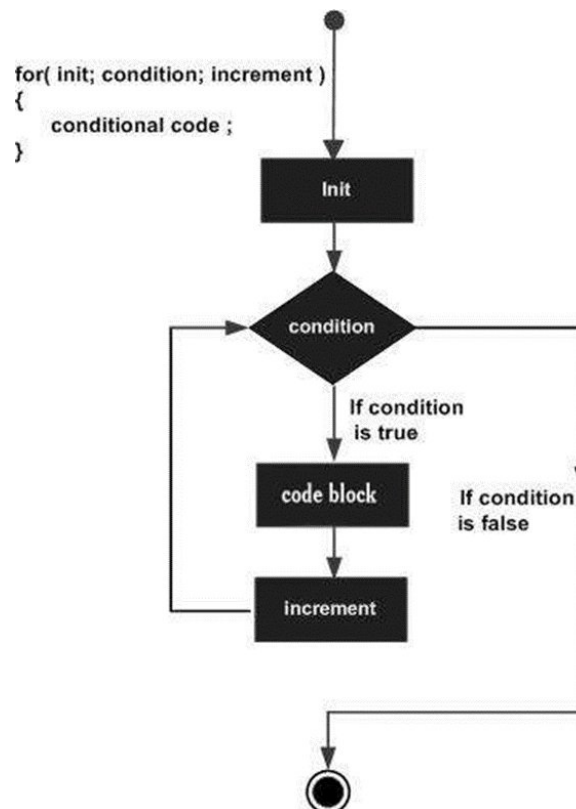
These loops are essential for performing repetitive tasks and iterating through data structures like arrays, and they provide different ways to control the flow of your program based on certain conditions.

5.2 FOR LOOP

The for loop in C++ is used when you know the number of iterations you want to perform in advance. It is a versatile loop that consists of three parts: initialization, condition, and increment/decrement. The loop continues to execute if the condition is true. Here's the basic structure of a for loop:

```
for (initialization; condition; increment/decrement) {
```

```
    // Code to be executed in each iteration }  
}
```



Let's look at an example of a for loop that counts from 1 to 5:

```
#include <iostream>

using namespace std;

int main() {

    for (int i = 1; i <= 5; i++) {

        cout << "Iteration " << i << endl;

    }

    cout << "End of program." << endl;

    return 0;

}
```

In this example:

`int i = 1` is the initialization part. It initializes the loop control variable `i` to 1 at the beginning of the loop.

`i <= 5` is the condition. If this condition is true (i.e., `i` is less than or equal to 5), the loop will continue to execute.

`i++` is the increment part. It increases the value of `i` by 1 after each iteration.

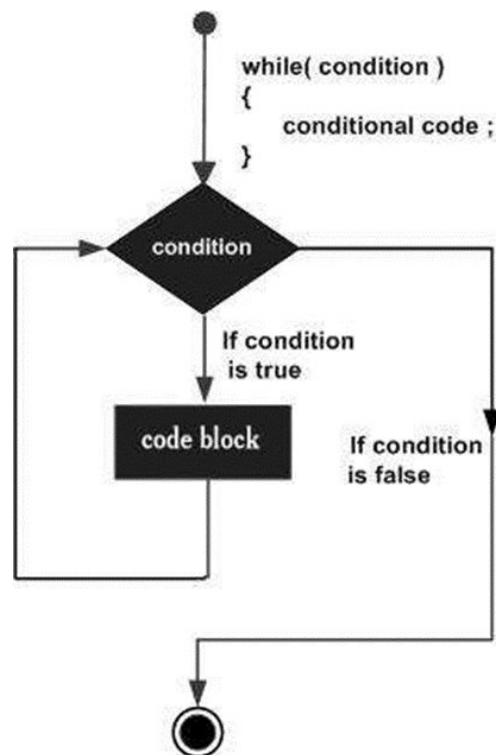
The loop will execute five times, as specified by the condition. In each iteration, it prints "Iteration 1," "Iteration 2," and so on, until it reaches "Iteration 5." After that, the loop terminates, and the program prints "End of program."

This is a simple example, but for loops are commonly used for a wide range of tasks, such as iterating over arrays, processing data, and performing repetitive calculations. The ability to control the number of iterations and the loop control variable makes for loops a powerful tool in C++.

5.3 WHILE LOOP

A while loop in C++ is used when you want to repeatedly execute a block of code if a certain condition is true. The condition is checked before each iteration, and if it's true, the loop continues to execute. Here's the basic structure of a while loop:

```
while (condition) {  
  
    // Code to be executed in each iteration  
  
}
```



Let's look at an example of a while loop that counts from 1 to 5:

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
  
    int i = 1; // Initialization  
  
    while (i <= 5) { // Condition
```

```
    cout << "Iteration " << i << endl;

    i++; // Increment

}

cout << "End of program." << endl;

return 0;

}
```

In this example:

- `int i = 1` is the initialization. It initializes the loop control variable `i` to 1 before entering the loop.
- `i <= 5` is the condition. The loop continues to execute if this condition is true (i.e., `i` is less than or equal to 5).
- `i++` is the increment part. It increases the value of `i` by 1 in each iteration.

The loop starts with `i` equal to 1, and it checks the condition. Since `i` is less than or equal to 5, the condition is true, and the loop executes. It prints "Iteration 1," increments `i` to 2, checks the condition again, and so on. The loop continues to execute until `i` becomes 6, at which point the condition is no longer true, and the loop terminates.

The output of the program will be:

Iteration 1

Iteration 2

Iteration 3

Iteration 4

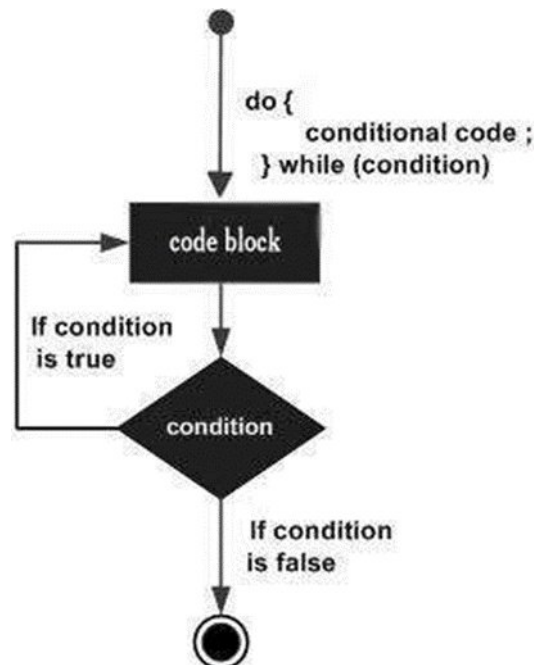
Iteration 5

End of program.

5.4 DO-WHILE LOOP

A do-while loop in C++ is like a while loop, but with a crucial difference: the condition is checked at the end of each iteration. This means that the loop's body is guaranteed to execute at least once, regardless of whether the condition is initially true or false. Here's the basic structure of a do-while loop.

```
do {  
  
    // Code to be executed in each iteration  
  
} while (condition);
```



Here's a breakdown of how a "do-while" loop works:

- The code block inside the "do" is executed.
- The condition in the "while" statement is evaluated.
- If the condition is true, the loop continues to the next iteration and repeats the steps. If the condition is false, the loop terminates, and the program continues with the code after the loop.

Let's see an example of a "do-while" loop in C++ that asks the user for a number and keeps asking until the user enters a positive number:

```
#include <iostream>

using namespace std;

int main() {

    int num;

    do {

        cout << "Enter a positive number: ";

        cin >> num;

        if (num <= 0) {

            cout << "Please enter a positive number." << endl;

        }

    } while (num <= 0);

    cout << "You entered a positive number: " << num << endl;

    return 0;

}
```

In this example:

- The program starts by declaring an integer variable num.
- The "do" block contains a prompt for the user to enter a positive number and reads the input into the num variable.
- It then checks if the entered number is less than or equal to zero. If it is, the program displays an error message and continues with the loop.
- The "while" statement checks if the condition num <= 0 is true. If it is true, the loop repeats, and the user is prompted to enter a number again. This process continues until the user enters a positive number.

- Once the user enters a positive number, the loop terminates, and the program displays a message with the entered number.

This demonstrates how a "do-while" loop is useful for situations where you want to ensure that a block of code is executed at least once, and then repeated based on a condition.

5.5 LOOP CONTROL STATEMENTS

Loop control statements are like special instructions that can change the usual order of actions in a program. When you finish a set of actions, any temporary things you made during those actions are removed.

In C++, you have different control statements for loops that let you change how loops work:

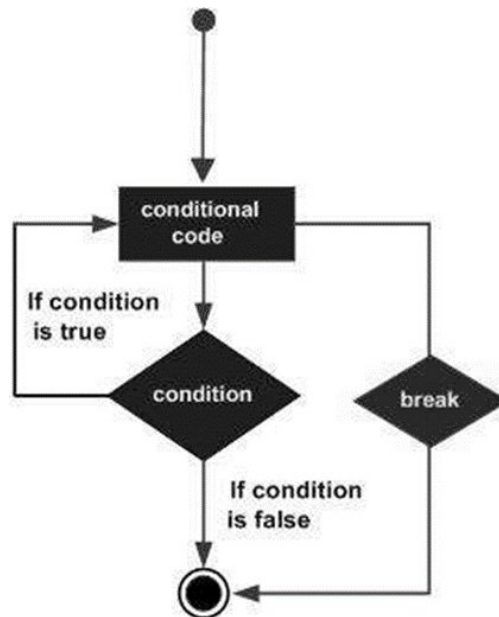
Control Statement	Description
break statement	Terminates the loop or switch statement and transfers execution to the statement immediately following the loop or switch.
continue statement	Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.
goto statement	Transfers control to the labeled statement. Though it is not advised to use goto statement in your program.

5.5.1 Break Statement:

The "break" statement in C++ serves two main purposes:

- If you use it inside a loop, it stops the loop right away. Your program then moves to the next statement after the loop.
- It's also used in "switch" statements, which we'll cover later. In that context, "break" ends a specific case.

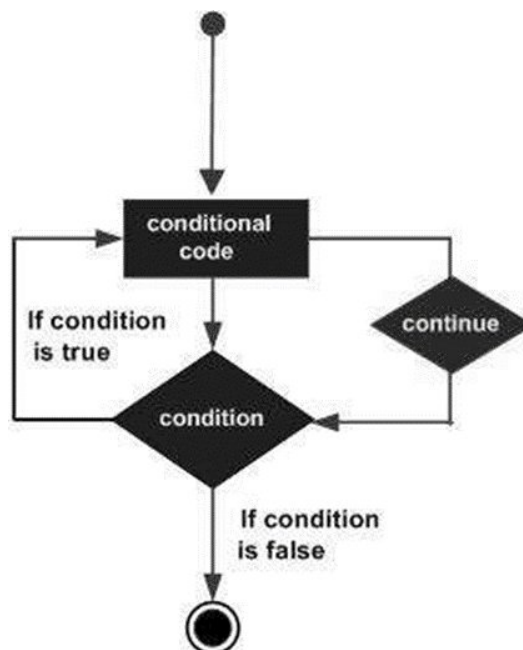
If you have loops inside other loops (nested loops), "break" will stop only the innermost loop, and the program continues with the next line of code outside that loop. Let us understand through the flow diagram about the working of the break statements in C++:



5.5.2 Continue Statement:

The continue statement is a bit like break, but not quite. Instead of ending the loop, it says, Let's move on to the next round of the loop right away and skip anything in between.

In a for loop, continue makes it jump to the part where it checks the condition and increases the counter. In while and do...while loops, it takes you back to check the conditions again. Let us understand better through the flow chart:



5.5.3 goto statement

A goto statement is like a big leap in your program, taking you directly to a labelled spot within the same part of your code.

But here's the thing: Using goto is not recommended because it can make your program confusing and messy. It's like a hidden trapdoor that can be hard to follow and change. Most of the time, you can find a better way to write your program without using goto.

Let us understand through the syntax:

```
goto label;
```

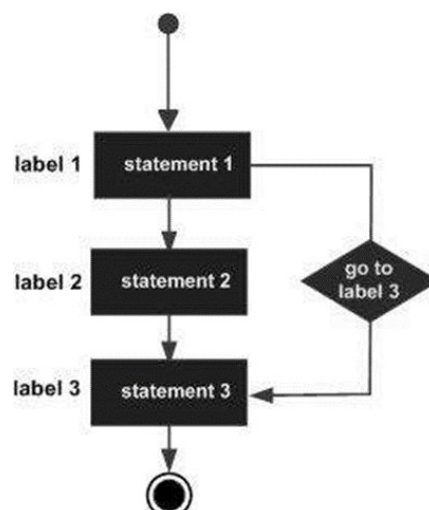
```
..
```

```
.
```

```
label: statement;
```

In this context, a label is like a name tag for a statement. It's just a word followed by a colon (:), and it helps you identify that statement later. It's a way to mark a specific spot in your code so you can refer to it easily.

Let us understand the goto statements using the flow chart diagram:



5.6 THE INFINITE LOOP

A loop turns into an infinite loop when its condition never becomes false. We often use the for loop for this. You can create an endless loop by simply leaving out the part where you check the condition. If you don't have a condition to check, it's like saying, Yeah, it's always true.

To make an endless loop, people often use "for (;;)" in C++. It's a way of saying, Keep going forever. And remember, if you ever want to stop an endless loop, you can press Ctrl + C. It's like hitting a pause button for your program.

5.7 SUMMARY

In this chapter, the focus is on comprehending the fundamental concept of loops in the context of C++ programming. Loops are introduced as structures that enable the repetition of tasks, enhancing the flexibility of a program. The three primary loop types in C++ — for, while, and do-while — are elucidated, each serving distinct purposes in repetitive execution. The for loop, characterized by initialization, condition, and increment/decrement components, is presented as a versatile tool for situations where the number of iterations is known in advance. A practical example reinforces the understanding of for loop implementation.

The subsequent sections delve into while and do-while loops, highlighting their distinct characteristics. The while loop is emphasized for scenarios requiring repetitive execution based on a specified condition, checked at the beginning of each iteration. In contrast, the do-while loop guarantees at least one execution of its body by evaluating the condition at the end of each iteration.

The chapter also introduces loop control statements—break, continue, and goto—which offer specialized instructions to alter the standard flow of a program. Break terminates a loop or switch statement, continue skips the remainder of a loop's body, and goto facilitates non-linear program control, though its use is discouraged.

5.8 KEYWORDS

Loop, for Loop, while Loop, do-while Loop, Loop Control Statements, break statement, continue statement, goto statement, Infinite Loop, Control Statement

5.9 EXERCISE

1. What is the purpose of a loop in programming?
 - a) To break the program execution
 - b) To execute a block of code repeatedly
 - c) To skip certain statements
 - d) To terminate the program
2. Which type of loop in C++ is used when you know the number of iterations you want to perform in advance?
 - a) while Loop
 - b) do-while Loop
 - c) for Loop
 - d) goto Loop
3. What are the three parts of a for loop in C++?
 - a) Initialization, condition, and decrement
 - b) Initialization, condition, and increment/decrement
 - c) Initialization, condition, and multiply
 - d) Initialization, compare, and increment
4. In a while loop, when is the condition checked?
 - a) At the end of each iteration
 - b) At the beginning of each iteration
 - c) Before entering the loop
 - d) After exiting the loop
5. Which loop guarantees that the loop body is executed at least once?
 - a) while Loop
 - b) for Loop
 - c) do-while Loop
 - d) goto Loop
6. What is the purpose of the break statement in C++?

- a) To skip the remainder of the loop body and retest the condition
 - b) To terminate the loop or switch statement and transfer execution
 - c) To cause the loop to repeat immediately
 - d) To jump to a labeled statement
7. How does the continue statement differ from the break statement?
- a) Continue ends the loop; break skips the loop body.
 - b) Continue immediately retests the loop condition; break terminates the loop.
 - c) Continue terminates the loop; break retests the loop condition.
 - d) Continue and break are used interchangeably.
8. Why is the use of the goto statement not recommended in C++?
- a) It is difficult to understand and can lead to confusing code.
 - b) It simplifies the control flow of the program.
 - c) It improves code readability.
 - d) It is a required element in all programs.
9. How is an infinite loop created in C++ using the for loop?
- a) for (;;)
 - b) for (int i = 0; i < 10;)
 - c) for (int i = 1; i <= 5; i++)
 - d) for (int i = 5; i > 0; i--)
10. How can you stop an endless loop in C++?
- a) Press Ctrl + Z
 - b) Press Ctrl + S
 - c) Press Ctrl + C
 - d) Press Ctrl + X
11. Write a program to calculate factorial of a number, inputted through keyboard.
12. Write a program to print 1st 20 members of FIBONACCI Series. The first few members are:
- 1 1 2 3 5 8 13

5.10 REFERENCES

- "The C++ Programming Language" by Bjarne Stroustrup:
 - Author: Bjarne Stroustrup (the creator of C++)
 - Description: This is considered the definitive book for learning C++. It covers the language in-depth and provides insights into its design and evolution.
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo:
 - Authors: Stanley B. Lippman, Josée Lajoie, Barbara E. Moo
 - Description: Known for its clear and concise style, this book is suitable for beginners and serves as an excellent reference for more advanced topics.
- "Effective C++: 55 Specific Ways to Improve Your Programs and Designs" by Scott Meyers:
 - Author: Scott Meyers
 - Description: This book provides practical advice and tips for writing better C++ code. It focuses on best practices and common pitfalls.
- "Accelerated C++: Practical Programming by Example" by Andrew Koenig and Barbara E. Moo:
 - Authors: Andrew Koenig, Barbara E. Moo
 - Description: Aimed at those new to programming, this book takes a practical approach to learning C++. It emphasizes writing real programs early in the learning process.
- "C++ Standard Library - A Tutorial and Reference" by Nicolai M. Josuttis:
 - Author: Nicolai M. Josuttis
 - Description: This book provides a comprehensive guide to the C++ Standard Library. It covers the containers, algorithms, and other components of the library, offering both tutorials and reference material.

UNIT 6: ARRAY

CONTENT:

- **Objectives**

- 6.1 Introduction**

- 6.2 One-Dimensional Array**

- 6.3 Two-Dimensional Array**

- 6.4 Strings**

- 6.5 Summary**

- 6.6 Keywords**

- 6.7 Exercise**

- 6.8 References**

OBJECTIVES:

After studying this unit, you will be able to:

- Comprehend the use of arrays to store elements of the same data type in contiguous memory.
- Learn key features like homogeneous elements, contiguous memory, fixed size, and indexed access.
- Demonstrate the declaration, initialization, and access of elements in C++ arrays for various data types.
- Grasp the concept of two-dimensional arrays, with examples of declaration, initialization, and element access.

6.1 ARRAY

In C++, there's a useful way to store a bunch of data called an array. It's like having a set of boxes to put things in. All the stuff in an array must be the same kind of thing, like numbers or words.

Instead of making separate boxes (variables) for each thing, like `number0`, `number1`, and so on, you can create one big box (array) called `numbers`. To pick out a specific thing from the box, you use a number called an index. For example, to get the first thing in the `numbers` box, you use `numbers [0]`, and for the second thing, you use `numbers [1]`. These boxes are all lined up in memory, from the first one to the last one. The first box is at the lowest memory address, and the last box is at the highest address. It's a convenient way to organize and work with a bunch of related data.

In C++, an array is a collection of elements of the same data type stored in contiguous memory locations. Each element in the array is accessed by its index, which is an integer value representing its position in the array. Arrays in C++ have a fixed size, which is determined when the array is declared, and this size cannot be changed during the program's execution.

There are Two types of array commonly used:

- **One-Dimensional Array:** A one-dimensional array is the most basic type of array and consists of elements stored in a single row or column. Elements are accessed using a single index.
- **Multi-Dimensional Array:** Multi-dimensional arrays are arrays with more than one dimension. Common examples include two-dimensional arrays (matrices) and three-dimensional arrays. These arrays are often used for representing tables or grids of data.

6.2 ONE-DIMENSIONAL ARRAY

A one-dimensional array is a fundamental data structure in C++ and other programming languages. It is a collection of elements of the same data type, stored in a linear sequence, with each element identified by a unique index. One-dimensional arrays are also often referred to simply as "arrays."

Key characteristics of one-dimensional arrays in C++:

- **Homogeneous Elements:** All elements in a one-dimensional array must be of the same data type. For example, you can have an array of integers, an array of floating-point numbers, or an array of characters.
- **Contiguous Memory:** The elements in a one-dimensional array are stored in contiguous memory locations. This means that they are placed one after the other in memory, making it efficient to access elements based on their indices.
- **Fixed Size:** One-dimensional arrays have a fixed size, which is determined when the array is declared. The size cannot be changed during program execution.
- **Indexed Access:** Elements in a one-dimensional array are accessed using integer indices. The first element is typically accessed using an index of 0, the second element with an index of 1, and so on. The last element is accessed using an index of size - 1, where size is the size of the array.

Here's how you declare and use an array in C++:

// Declaration and initialization of an integer array

```
int myArray[5];           // This creates an array of 5 integers
```

// Initializing the elements of the array

```
myArray[0] = 1;
```

```
myArray[1] = 2;
```

```
myArray[2] = 3;
```

```
myArray[3] = 4;
```

```
myArray[4] = 5;
```

// Accessing elements of the array

```
int element = myArray[2]; // Accessing the third element (index 2), which is 3
```

```
// You can also declare and initialize an array in one line
```

```
int anotherArray[] = {10, 20, 30, 40, 50}; // The size is determined by the number of elements
```

```
// Arrays can hold elements of any data type
```

```
double doubleArray[3] = {1.1, 2.2, 3.3};
```

```
// Finding the length (number of elements) of an array
```

```
int length = sizeof(myArray) / sizeof(myArray[0]); // Gives you the length, which is 5 in this case
```

Here's an example of a one-dimensional array in C++. In this example, we'll create an array to store and display a list of integers:

```
#include <iostream>
```

```
Using namespace std;
```

```
int main() {
```

```
    // Declare and initialize a one-dimensional array of integers
```

```
    int numbers[5] = {10, 20, 30, 40, 50};
```

```
    // Access and display the elements of the array
```

```
    std::cout << "Elements of the array:" << std::endl;
```

```
    for (int i = 0; i < 5; i++) {
```

```
        cout << "Element at index " << i << ": " << numbers[i] << std::endl;
```

```
    }
```



```
    return 0;  
  
}
```

In this example, we:

1. Declare and initialize a one-dimensional integer array called numbers with 5 elements.
2. Use a for loop to iterate through the array, accessing and displaying each element.
3. Print the index and value of each element to the console.

When you run this program, it will display the elements of the array:

Elements of the array:

Element at index 0: 10

Element at index 1: 20

Element at index 2: 30

Element at index 3: 40

Element at index 4: 50

6.3 TWO-DIMENSIONAL ARRAY

A two-dimensional array, often referred to as a matrix, is a data structure in C++ that consists of elements organized in rows and columns, forming a grid-like structure. It is used to represent tables, grids, or matrices of data where elements can be accessed using two indices: one for the row and one for the column.

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Here's how you declare and work with a two-dimensional array in C++:

// Declare and initialize a 2D array

```
int matrix[3][4] = {
```

```
    {1, 2, 3, 4},
```

```
    {5, 6, 7, 8},
```

```
    {9, 10, 11, 12}
```

```
};
```

// Accessing elements of the 2D array

```
int element = matrix[1][2]; // Accessing the element in the second row and third column (7)
```

// Loop to iterate through the elements of the 2D array

```
for (int row = 0; row < 3; row++) {
```

```
    for (int col = 0; col < 4; col++) {
```

```
        cout << matrix[row][col] << " ";
```

```
    }
```

```
    cout << std::endl;
```

```
}
```

Here's an example of a two-dimensional array in C++ that represents a simple matrix, and then we'll demonstrate how to access its elements and print them:

In this example, we:

1. Declare and initialize a 2D integer array called matrix with 3 rows and 4 columns.
2. Use nested for loops to iterate through the elements of the array. The outer loop iterates over rows (from 0 to 2), and the inner loop iterates over columns (from 0 to 3).
3. We access and print each element of the matrix using `matrix[row][col]`, where row represents the row index, and col represents the column index.

```
#include <iostream>

using namespace std;

int main() {

    // Declare and initialize a 2D array (a 3x4 matrix)

    int matrix[3][4] = {

        {1, 2, 3, 4},

        {5, 6, 7, 8},

        {9, 10, 11, 12}

    };

    // Access and display elements of the 2D array

    cout << "Elements of the 2D array:" << endl;

    for (int row = 0; row < 3; row++) {

        for (int col = 0; col < 4; col++) {

            cout << matrix[row][col] << " ";
```

```
    }  
  
    cout << endl; // Move to the next row  
  
    }  
  
    return 0;  
  
}
```

The result is a display of the elements of the 2D array in a matrix-like format:

Elements of the 2D array:

```
1 2 3 4  
  
5 6 7 8  
  
9 10 11 12
```

6.4 STRINGS

C++ provides two ways to work with strings:

1. C-style strings are represented as arrays of characters, terminated by a null character (`\0`).
2. C++ strings are represented by the `std::string` class.

C-style strings are simpler to use, but they are also more error-prone. C++ strings are safer and more powerful, but they are also more complex to use.

To declare a C-style string, you need to specify the size of the character array. For example:

C++

```
char my_string[10];
```

This will declare a character array with 10 elements. This is enough space to store a string of up to 9 characters, plus the null character.

To initialize a C-style string, you can use the `strcpy()` function. For example:

C++

```
strcpy(my_string, "Hello");
```

Use code with caution. [Learn more](#)

This will copy the string "Hello" into the `my_string` array.

To access individual characters in a C-style string, you can use the `[]` operator. For example:

C++

```
char first_char = my_string[0];
```

This will assign the first character of the `my_string` array to the variable `first_char`.

To get the length of a C-style string, you can use the `strlen()` function. For example:

C++

```
int length = strlen(my_string);
```

This will assign the length of the `my_string` array to the variable `length`.

Working with C++ strings

To declare a C++ string, you can use the `std::string` class. For example:

C++

```
std::string my_string = "Hello";
```

This will create a new `std::string` object and initialize it with the string "Hello".

To access individual characters in a C++ string, you can use the `[]` operator. For example:

C++

```
char first_char = my_string[0];
```

This will assign the first character of the `my_string` object to the variable `first_char`.

To get the length of a C++ string, you can use the `length()` member function. For example:

C++

```
int length = my_string.length();
```

This will assign the length of the `my_string` object to the variable `length`.

Here's a C++ program that takes input from the user, processes the input string, and counts the number of vowels in the input string:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main() {
```

```
    string inputString;
```

```
    cout << "Enter a string: ";
```

```
    getline(cin, inputString); // Read the entire line, including spaces
```

```
    int vowelCount = 0;
```

```
    for (char c : inputString) {
```

```
        // Convert the character to lowercase for case-insensitive comparison
```

```
        char lowercaseChar = tolower(c);
```

```
        // Check if the character is a vowel
```

```
        if (lowercaseChar == 'a' || lowercaseChar == 'e' || lowercaseChar == 'i' || lowercaseChar ==  
'o' || lowercaseChar == 'u') {
```

```
            vowelCount++;
```

```
    }  
  
}  
  
cout << "Number of vowels in the string: " << vowelCount << endl;  
  
return 0;  
  
}
```

In this program:

1. We declare a string `inputString` to store the user's input.
2. We use `getline` to read the entire line of input, including spaces.
3. We initialize a counter `vowelCount` to 0 to keep track of the number of vowels.
4. We iterate through each character in the `inputString`. To make the comparison case-insensitive, we convert each character to lowercase using the `tolower` function.
5. We check if the lowercase character is one of the vowels ('a', 'e', 'i', 'o', 'u') and increment the `vowelCount` when a vowel is found.
6. Finally, we print the number of vowels in the input string.

This program will count both uppercase and lowercase vowels as vowels and provide the total count.

6.5 SUMMARY

In C++, arrays serve as a fundamental data structure for storing elements of the same data type in contiguous memory. This unit provides a comprehensive understanding of arrays, emphasizing their role in efficient data organization. Key features of one-dimensional arrays are explored, including the requirement for homogeneous elements, contiguous memory storage, fixed size determination during declaration, and indexed access for element retrieval.

The unit delves into the practical aspects of array manipulation in C++, offering insights into the declaration, initialization, and access of array elements. It covers diverse scenarios, showcasing individual assignments and concise one-line declarations. Examples span various data types, demonstrating the versatility of arrays in accommodating different kinds of information.

Furthermore, the unit introduces the concept of two-dimensional arrays in C++, presenting them as grid-like structures with rows and columns. Examples elucidate the declaration, initialization, and accessing of elements in a two-dimensional array. Overall, this unit serves as a foundational guide for mastering arrays in C++, enabling learners to comprehend, apply, and appreciate the efficiency and versatility these structures bring to the realm of programming.

6.6 KEYWORDS

Array, One-Dimensional Array, Multi-Dimensional Array, Homogeneous Elements, Contiguous Memory, Fixed Size, Indexed Access, C-style Strings, `std::string`, `getline()`

6.7 EXERCISE

1. What term refers to a collection of elements of the same data type stored in contiguous memory locations?
 - a. Homogeneous Elements
 - b. Contiguous Memory
 - c. Indexed Access
 - d. C-style Strings
2. Which keyword is associated with arrays and is used for accessing elements based on their position?
 - a. Contiguous Memory
 - b. Indexed Access
 - c. One-Dimensional Array
 - d. Fixed Size
3. What is a fundamental data structure in C++ consisting of elements stored in a linear sequence, with each element identified by a unique index?
 - a. Multi-Dimensional Array
 - b. One-Dimensional Array
 - c. `getline`
 - d. `std::string`
4. Which characteristic is true for C++ arrays and ensures that elements are stored one after the other in memory?
 - a. Homogeneous Elements
 - b. Contiguous Memory
 - c. Fixed Size

- d. C-style Strings
5. In C++, which type of array represents a grid-like structure with elements organized in rows and columns?
 - a. One-Dimensional Array
 - b. Multi-Dimensional Array
 - c. Fixed Size
 - d. `std::string`
6. What is an array in C++ and how is it useful for storing data?
7. Explain the concept of an index in the context of an array. How is it used to access elements?
8. Differentiate between a one-dimensional array and a multi-dimensional array in C++. Provide examples for each.
9. What are the key characteristics of a one-dimensional array in C++?
10. In C++, how do you declare and initialize a one-dimensional array with 5 integer elements?
11. Explain the purpose and use of the `sizeof` operator in finding the length of an array.
12. How do you declare and initialize a two-dimensional array in C++? Provide an example.
13. What are the advantages of using C++ strings over C-style strings?
14. How do you access individual characters in a C++ string? Provide an example.
15. Write a C++ program that takes a user input string, processes it, and counts the number of vowels (both uppercase and lowercase) in the string.

6.8 REFERENCES

- "The C++ Programming Language" by Bjarne Stroustrup:
 - Author: Bjarne Stroustrup (the creator of C++)
 - Description: This is considered the definitive book for learning C++. It covers the language in-depth and provides insights into its design and evolution.
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo:
 - Authors: Stanley B. Lippman, Josée Lajoie, Barbara E. Moo
 - Description: Known for its clear and concise style, this book is suitable for beginners and serves as an excellent reference for more advanced topics.
- "Effective C++: 55 Specific Ways to Improve Your Programs and Designs" by Scott Meyers:

- Author: Scott Meyers
 - Description: This book provides practical advice and tips for writing better C++ code. It focuses on best practices and common pitfalls.
- "Accelerated C++: Practical Programming by Example" by Andrew Koenig and Barbara E. Moo:
 - Authors: Andrew Koenig, Barbara E. Moo
 - Description: Aimed at those new to programming, this book takes a practical approach to learning C++. It emphasizes writing real programs early in the learning process.
- "C++ Standard Library - A Tutorial and Reference" by Nicolai M. Josuttis:
 - Author: Nicolai M. Josuttis
 - Description: This book provides a comprehensive guide to the C++ Standard Library. It covers the containers, algorithms, and other components of the library, offering both tutorials and reference material.

UNIT 7: FUNCTIONS

CONTENT:

▪ Objectives

7.1 Introduction

7.2 Function Declaration

7.3 Function Definition

7.4 Function Call

7.5 Return Statement

7.6 Parameters and Arguments

7.7 Function Prototypes

7.8 Recursion

7.9 Summary

7.10 Keywords

7.11 Exercise

7.12 References

OBJECTIVES:

After studying this unit, you will be able to:

- Learn the concept and significance of functions in structured programming.
- Grasp the process of declaring functions, specifying return type, name, and parameters.
- Learn the elements of a function definition, including name, return type, and parameters.
- Explore calling functions, using return statements, and implementing functions in a C++ program.

7.1 INTRODUCTION

In C++, a function is a self-contained block of code that performs a specific task or a set of related tasks. Functions are an essential part of structured programming and help in organizing and modularizing code. Functions provide several advantages, including code reusability, better maintainability, and easier debugging. Here are the key characteristics and elements of functions in C++:

7.2 FUNCTION DECLARATION

You start by declaring a function, specifying its return type, name, and any parameters it accepts. For example:

```
int add(int a, int b);
```

7.3 FUNCTION DEFINITION

The function definition provides the actual implementation of the function. It includes the function's name, return type, and parameter list, followed by the function body enclosed in curly braces. For example:

```
int add(int a, int b) {  
  
    return a + b;  
  
}
```

7.4 FUNCTION CALL

You call a function by using its name, passing the required arguments (if any), and optionally using the return value. For example:

```
int result = add(5, 3);
```

7.5 RETURN STATEMENT

Functions can return a value using the `return` statement. The type of the return value must match the function's declared return type. For example:

```
int add(int a, int b) {  
  
    return a + b; }
```

7.6 PARAMETERS AND ARGUMENTS

Parameters are variables declared in the function's parameter list and are used to accept values passed during the function call. Arguments are the actual values passed to the function when it's called. For example:

```
int add(int a, int b) {  
  
    // 'a' and 'b' are parameters  
  
    return a + b; }  
  
int result = add(5, 3); // 5 and 3 are arguments
```

7.7 FUNCTION PROTOTYPES

It's common to declare a function prototype before using a function. The function prototype provides the function's signature (return type, name, and parameter list) without the implementation details. This allows you to use the function before defining it.

Functions are a fundamental building block of C++ programs, allowing for better code organization, reusability, and abstraction. They are used extensively in C++ to create structured and efficient programs.

Here's a simple C++ program that calls a function to add two numbers:

```
#include <iostream>  
  
using namespace std;  
  
// Function to add two numbers  
  
int add(int a, int b) {  
  
    return a + b;
```

```
}

int main() {

    int num1, num2;

    // Input the two numbers

    cout << "Enter the first number: ";

    cin >> num1;

    cout << "Enter the second number: ";

    cin >> num2;

    // Call the 'add' function and store the result in 'sum'

    int sum = add(num1, num2);

    // Display the result

    cout << "The sum of " << num1 << " and " << num2 << " is: " << sum << endl;

    return 0; }
```

In this program:

1. We declare a function add that takes two integer parameters and returns their sum.
2. In the main function, we declare two integer variables num1 and num2 to store the user's input.
3. We use cin to get input from the user for both numbers.
4. We call the add function with num1 and num2 as arguments, and the result is stored in the sum variable.
5. Finally, we use cout to display the result, which is the sum of the two numbers.

When you run the program, it will prompt you to enter two numbers, and then it will display the sum of those numbers.

Functions in C++ are used for several important reasons:

- **Modularization and Code Reusability:** Functions allow you to break down a program into smaller, manageable, and reusable pieces. You can write a function to perform a specific task and use it multiple times in your program or even in other programs. This promotes code reusability and simplifies maintenance.
- **Abstraction:** Functions provide an abstraction layer that hides the implementation details. When you call a function, you don't need to know how it accomplishes the task; you only need to know its interface (the function signature).
- **Structured Programming:** Functions are fundamental to structured programming, which promotes organized, readable, and maintainable code. You can divide your program into smaller functions, each responsible for a specific aspect of the task.
- **Separation of Concerns:** Functions allow you to separate different concerns or responsibilities in your program. For example, you can have separate functions for input, processing, and output, making it easier to understand and modify each part independently.
- **Testing and Debugging:** Functions simplify the testing and debugging process. You can test and debug individual functions in isolation, making it easier to identify and fix issues. This also supports unit testing.
- **Efficiency:** Functions promote code efficiency. You can avoid duplicating code by encapsulating common operations in functions. Additionally, functions can be optimized separately.
- **Readability:** Well-named functions make the code more readable. A function name can describe the purpose of the operation it performs, making the code self-documenting.
- **Encapsulation:** Functions can have their own local variables, which are hidden from the rest of the program. This encapsulation allows you to isolate variables and avoid naming conflicts.

In summary, functions in C++ provide a structured and efficient way to design, organize, and maintain your code. They encourage good programming practices, enhance code readability, and support code reuse, making your programs more maintainable and scalable.

7.8 RECURSION

Recursion in C++ is a programming technique in which a function calls itself to solve a problem. It's a powerful and elegant way to solve problems that can be divided into smaller, similar subproblems. Recursion is based on the principle of self-reference, where a function reduces a problem into a simpler version of the same problem until a base case is reached, which provides a solution without further recursive calls.

Key elements of recursion in C++:

1. **Base Case:** Every recursive function must have one or more base cases. The base case is a condition that, when met, terminates the recursion. It provides the result without further recursive calls, preventing infinite recursion.
2. **Recursive Case:** In the recursive case, the function calls itself with a modified input to solve a smaller or simpler version of the problem. The function typically combines the result of the current operation with the result of the recursive call.
3. **Recursive Function Call:** The function calls itself within its own body. This is the defining characteristic of recursive functions.

Here's an example of a recursive function in C++ that calculates the factorial of a non-negative integer:

```
#include <iostream>
```

```
Using namespace std;
```

```
int factorial(int n) {
```

```
    // Base case
```

```
    if (n == 0 || n == 1) {
```

```
        return 1;
```

```
    } else {
```

```
        // Recursive case
```



```
        return n * factorial(n - 1);

    }

}

int main() {

    int num;

    cout << "Enter a non-negative integer: ";

    cin >> num;

    if (num < 0) {

        cout << "Factorial is not defined for negative numbers." << endl;

    } else {

        int result = factorial(num);

        cout << "Factorial of " << num << " is " << result << endl;

    }

    return 0; }
```

In this example, the factorial function is recursive. It calculates the factorial of a number n by calling itself with $n - 1$ until it reaches the base case where n is 0 or 1. The base case returns a value of 1, and in the recursive case, the function multiplies n by the result of the recursive call with $n - 1$. This way, it builds the factorial by multiplying the numbers from n down to 1. Recursion is a powerful technique but should be used with care, as it can lead to stack overflow errors if not managed properly. It's important to ensure that there's a clear base case, and the input moves closer to the base case with each recursive call.

7.9 SUMMARY

In C++, functions serve as self-contained blocks of code, performing specific tasks and aiding in structured programming. They contribute to code organization, offering benefits such as

code reusability, maintainability, and ease of debugging. The process begins with function declaration, specifying return type, name, and parameters. Subsequently, the function definition encapsulates the actual implementation, comprising the function's name, return type, parameter list, and the code enclosed in curly braces. Function calls involve using the function name and passing arguments, with the optional use of return values.

Functions play a pivotal role in modularizing and enhancing code reusability. They provide abstraction by concealing implementation details, contributing to structured programming practices that facilitate organized and readable code. The separation of concerns is achieved through functions, allowing distinct responsibilities in a program. Functions also streamline testing and debugging, supporting unit testing and overall code efficiency. Their usage promotes readability, encapsulation, and the creation of well-organized programs.

The discussion extends to function prototypes, illustrating the importance of declaring a function's signature before its use. A sample C++ program is presented, demonstrating how functions add to program structure and efficiency. The study concludes with an introduction to recursion, highlighting its role in solving complex problems through self-referential function calls and emphasizing the importance of defining base and recursive cases. The recursive factorial example showcases the elegance and power of recursion in C++, albeit with a cautionary note on potential stack overflow errors if not managed carefully.

7.10 KEY WORDS

Function, Declaration, Definition, Call, Return Statement, Parameters, Arguments, Function Prototypes, Recursion

7.11 EXERCISE

1. What is a function in C++, and what purpose does it serve in programming?
2. Explain the process of declaring a function in C++. Provide an example.
3. What does the function definition in C++ include? Illustrate with an example.
4. How do you call a function in C++, and what components are involved in the process?
5. What is the significance of the return statement in C++ functions? Give an example.
6. Differentiate between parameters and arguments in the context of C++ functions. Provide an example.
7. Why is it common to use function prototypes in C++? How do they contribute to code organization?

8. Summarize the key reasons for using functions in C++ programming, as mentioned in the text.
9. Describe the role of recursion in C++ programming. What are its key elements?
10. Provide an example of a recursive function in C++ and explain how it works, emphasizing the base case and recursive case.

7.12 REFERENCES

- "The C++ Programming Language" by Bjarne Stroustrup:
 - Author: Bjarne Stroustrup (the creator of C++)
 - Description: This is considered the definitive book for learning C++. It covers the language in-depth and provides insights into its design and evolution.
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo:
 - Authors: Stanley B. Lippman, Josée Lajoie, Barbara E. Moo
 - Description: Known for its clear and concise style, this book is suitable for beginners and serves as an excellent reference for more advanced topics.
- "Effective C++: 55 Specific Ways to Improve Your Programs and Designs" by Scott Meyers:
 - Author: Scott Meyers
 - Description: This book provides practical advice and tips for writing better C++ code. It focuses on best practices and common pitfalls.
- "Accelerated C++: Practical Programming by Example" by Andrew Koenig and Barbara E. Moo:
 - Authors: Andrew Koenig, Barbara E. Moo
 - Description: Aimed at those new to programming, this book takes a practical approach to learning C++. It emphasizes writing real programs early in the learning process.
- "C++ Standard Library - A Tutorial and Reference" by Nicolai M. Josuttis:
 - Author: Nicolai M. Josuttis
 - Description: This book provides a comprehensive guide to the C++ Standard Library. It covers the containers, algorithms, and other components of the library, offering both tutorials and reference material.

UNIT 8: POINTERS

CONTENT:

- **Objectives**
 - 8.1 Introduction**
 - 8.2 Declaration and Initialization**
 - 8.3 Dereferencing**
 - 8.4 Pointer Arithmetic**
 - 8.5 Null Pointers**
 - 8.6 Pointer to Functions**
 - 8.7 Dynamic Memory Allocation**
 - 8.8 Passing by Reference**
 - 8.9 Summary**
 - 8.10 Keywords**
 - 8.11 Exercise**
 - 8.12 References**

OBJECTIVES:

After studying this unit, you will be able to:

- Comprehend the concept of memory addresses in C++.
- Emphasize Optimization: Recognize the importance of optimizing memory usage.
- The means of accessing and manipulating data stored in memory.
- Familiarize with advanced features like dynamic memory allocation in C++.

8.1 INTRODUCTION

Pointers in C++ are variables that store memory addresses. They are a fundamental and powerful feature of the C++ language, allowing you to work directly with memory locations, manipulate data, and optimize memory usage. Pointers are often used for tasks like dynamic memory allocation, passing parameters to functions by reference, and working with arrays.

Here are some key concepts related to pointers in C++:

8.2 DECLARATION AND INITIALIZATION

To declare a pointer, you use the data type followed by an asterisk (*). You can initialize a pointer with the address of another variable using the address-of operator (&), or you can initialize it to nullptr (a null pointer) or any valid memory address.

```
include <iostream>

using namespace std;

int main()

{

    // declare variables

    int var1 = 3;

    int var2 = 24;

    int var3 = 17;

    // print address of var1

    cout << "Address of var1: "<< &var1 << endl;

    // print address of var2

    cout << "Address of var2: " << &var2 << endl;

    // print address of var3
```

```
    cout << "Address of var3: " << &var3 << endl;

}
```

The output of above program will be:

Address of var1: 0x7fff5fbff8ac

Address of var2: 0x7fff5fbff8a8

Address of var3: 0x7fff5fbff8a4

Here, 0x at the beginning represents the address is in the hexadecimal form. Notice that the first address differs from the second by 4 bytes and the second address differs from the third by 4 bytes. This is because the size of an int variable is 4 bytes in a 64-bit system.

As mentioned above, pointers are used to store addresses rather than values. Here is how we can declare pointers.

```
int* intPointer;    // Declaration of an integer pointer
```

```
int x = 10;
```

```
int* pX = &x;      // Initializing with the address of x
```

```
int* nullPtr = nullptr; // Initializing with a null pointer
```

8.3 DEREFERENCING

To access the value at the memory location pointed to by a pointer, you use the dereference operator (*).

```
int y = *pX; // Dereferencing pX to get the value stored at the address it points to
```

```
#include <iostream>
```

```
using namespace std;
```

```

int main() {

    int var = 5;

    // declare pointer variable

    int* pointVar;

    // store address of var

    pointVar = &var;

    // print value of var

    cout << "var = " << var << endl;

    // print address of var

    cout << "Address of var (&var) = " << &var << endl

    << endl;

    // print pointer pointVar

    cout << "pointVar = " << pointVar << endl;

    // print the content of the address pointVar points to

    cout << "Content of the address pointed to by pointVar (*pointVar) = " << *pointVar <<
endl;

    return 0;

}

```

The Output of above code is :

var = 5

Address of var (&var) = 0x61ff08

```
pointVar = 0x61ff08
```

Content of the address pointed to by pointVar (*pointVar) = 5

In the above code, the address of var is assigned to pointVar. We have used the *pointVar to get the value stored in that address.

When * is used with pointers, it's called the dereference operator. It operates on a pointer and gives the value pointed by the address stored in the pointer. That is, *pointVar = var.

8.4 POINTER ARITHMETIC

You can perform arithmetic operations on pointers, such as incrementing and decrementing, to move between memory locations. Pointer arithmetic is particularly useful when working with arrays.

```
int arr[5] = {10, 20, 30, 40, 50};

int* pArr = arr; // Point to the first element

int firstElement = *pArr; // Access the first element

pArr++; // Move to the next element
```

8.5 NULL POINTERS

A null pointer points to no memory location. It's often used to indicate that a pointer does not currently point to a valid object. Dereferencing a null pointer leads to undefined behavior, so it should be checked before use.

```
int* pNull = nullptr;

if (pNull == nullptr) {

    // Check if the pointer is null

}
```


8.6 POINTER TO FUNCTIONS

Pointers can also point to functions. Function pointers allow you to select and call functions at runtime, making them useful in scenarios like callback functions or event.

```
int (*functionPtr)(int, int); // Pointer to a function that takes two int parameters and returns an int.
```

Let us have an example which demonstrate a function with and without a pointer:

```
#include <iostream>

using namespace std;

// Function to increment a value using call by value

void incrementByValue(int num) {

    num++;

}

int main() {

    int num = 5;

    cout << "Before incrementByValue: " << num << endl;

    // Calling the function with call by value

    incrementByValue(num);

    cout << "After incrementByValue: " << num << endl;

    return 0;

}
```

In this example, `incrementByValue` takes an integer `num` by value. The function increments its own copy of `num`, but the original variable `num` in the main function remains unchanged because the function works with a copy.

Now try same with pointer:

```
#include <iostream>

using namespace std;

// Function to increment a value using call by reference

void incrementByReference(int &num) {

    num++;

}

int main() {

    int num = 5;

    cout << "Before incrementByReference: " << num << endl;

    // Calling the function with call by reference

    incrementByReference(num);

    cout << "After incrementByReference: " << num << endl;

    return 0;

}
```

In this example, `incrementByReference` takes an integer `num` by reference using the `&` symbol. The function directly operates on the original variable `num` from the `main` function, so the changes made inside the function are reflected outside.

In the "call by reference" example, the function modifies the original variable, while in "call by value," the function only works with a copy of the variable, leaving the original unchanged.

8.7 DYNAMIC MEMORY ALLOCATION

Pointers are commonly used with dynamic memory allocation functions like `new` and `delete` or `malloc` and `free` in C. They allow you to allocate and deallocate memory at runtime.

```
int* dynamicArray = new int[10]; // Dynamically allocate an integer array
```

```
delete[] dynamicArray; // Deallocate the memory when done
```

8.8 PASSING BY REFERENCE

Pointers are used to pass variables by reference to functions, allowing the function to modify the original variable.

```
void modifyValue(int* pValue) {
```

```
    *pValue = 42;
```

```
}
```

```
int number = 10;
```

```
modifyValue(&number);
```

Pointers can be a powerful tool in C++, but they require careful management to avoid common pitfalls like dereferencing null pointers, memory leaks, and pointer arithmetic errors. Modern C++ also provides safer alternatives, like smart pointers, to help manage dynamic memory more easily and safely.

You can swap two numbers using a user-defined function with pointers in C++. Here's a program that demonstrates how to do this:

```
#include <iostream>
```

```
using namespace std;
```

```
// Function to swap two numbers using pointers
```

```
void swapNumbers(int* a, int* b) {
```

```
int temp = *a;

*a = *b;

*b = temp;

}

int main() {

    int num1, num2;


    // Input two numbers

    cout << "Enter the first number: ";

    cin >> num1;

    cout << "Enter the second number: ";

    cin >> num2;

    // Call the swapNumbers function to swap the values of num1 and num2

    swapNumbers(&num1, &num2);


    // Display the swapped values

    cout << "After swapping:" << endl;

    cout << "First number: " << num1 << endl;

    cout << "Second number: " << num2 << endl;

    return 0;

}
```

In this program:

1. We declare a user-defined function called `swapNumbers` that takes two integer pointers (`int*`) as parameters. This function swaps the values of the two numbers using a temporary variable.
2. In the main function, we input two numbers, `num1` and `num2`, from the user.
3. We call the `swapNumbers` function and pass the addresses of `num1` and `num2` as arguments (i.e., `&num1` and `&num2`) to swap their values using pointers.
4. Finally, we display the swapped values of `num1` and `num2`.

Using pointers allows the function to directly modify the original values of `num1` and `num2`.

8.9 SUMMARY

In this unit, the focus is on comprehending key aspects of C++ programming, particularly related to memory management and optimization. The first objective delves into the foundational understanding of memory addresses in C++ and underscores their pivotal role in programming efficiency. Following this, the second objective introduces the process of declaring and initializing variables in C++, providing a basis for effective coding.

Moving forward, the third objective sheds light on data access in C++, elucidating how to retrieve and manipulate information stored in memory. Lastly, the fourth objective explores advanced functionalities, such as dynamic memory allocation, presenting a glimpse into the more intricate aspects of C++ programming. Throughout these objectives, the aim is to equip learners with practical knowledge and coding techniques, emphasizing the significance of optimizing memory usage for efficient and effective C++ programming.

8.10 KEYWORDS

Memory addresses, Declaration and Initialization, Address-of operator (`&`), Dereferencing, dereference operator (`*`), Pointer Arithmetic, Null Pointers, Pointer to Functions, Dynamic Memory Allocation, `new` and `delete`, `malloc`, Passing by Reference

8.11 EXERCISE

1. What is a pointer in C++, and how does it differ from a regular variable?
2. Explain the process of declaring and initializing a pointer in C++ with examples.

3. How do you use the address-of operator to obtain the memory address of a variable in C++?
4. What does dereferencing mean in the context of pointers? Provide an example.
5. How can pointer arithmetic be useful when working with arrays in C++? Illustrate with an example.
6. What is a null pointer, and why is it used in C++? Explain the importance of checking for null pointers before dereferencing.
7. How are pointers used in dynamic memory allocation, and what functions are commonly associated with it?
8. What is the purpose of a pointer to a function in C++? Provide an example demonstrating its usage.
9. Discuss the concept of passing by reference using pointers in C++. How does it differ from passing by value?
10. Explain the potential pitfalls associated with using pointers in C++, and mention some safer alternatives available in modern C++ programming.

MCQ's

1. What is a pointer in C++?
 - a) A data type
 - b) A variable that stores memory addresses
 - c) A reserved keyword
 - d) A mathematical operator
2. How do you declare a pointer in C++?
 - a) pointer variable;
 - b) int* pointer;
 - c) ptr int;
 - d) point var = &variable;
3. What is the purpose of the address-of operator (&) when working with pointers?
 - a) It initializes a pointer.
 - b) It dereferences a pointer.
 - c) It obtains the memory address of a variable.
 - d) It performs pointer arithmetic.
 - e)

4. What is the significance of nullptr in C++ pointers?
 - a) It is a reserved keyword for pointer arithmetic.
 - b) It is a null pointer, indicating that a pointer does not point to a valid object.
 - c) It is used to declare a pointer.
 - d) It is a mathematical operator for pointer operations.
5. What does the dereference operator (*) do in C++ pointers?
 - a) It increments the pointer value.
 - b) It decrements the pointer value.
 - c) It accesses the value at the memory location pointed to by a pointer.
 - d) It checks if a pointer is null.
6. How can pointer arithmetic be useful in C++?
 - a) It allows you to perform mathematical operations on pointers.
 - b) It helps with dynamic memory allocation.
 - c) It simplifies the process of declaring pointers.
 - d) It facilitates the manipulation of data in arrays.
7. What is the role of null pointers, and why should they be checked before dereferencing?
 - a) Null pointers indicate a successful memory allocation.
 - b) Null pointers prevent pointer arithmetic errors.
 - c) Null pointers signify uninitialized variables.
 - d) Dereferencing a null pointer leads to undefined behavior.
8. How are pointers commonly used in dynamic memory allocation?
 - a) Pointers are used to declare static arrays.
 - b) Pointers are used to allocate and deallocate memory at runtime.
 - c) Pointers are used for mathematical calculations.
 - d) Pointers are used for conditional statements.
9. What does a pointer to a function allow you to do in C++?
 - a) It allows you to create mathematical functions.
 - b) It allows you to dynamically allocate functions.
 - c) It allows you to select and call functions at runtime.
 - d) It allows you to perform arithmetic operations on functions.
10. What potential pitfalls are associated with using pointers in C++?
 - a) Memory leaks, null pointer errors, and arithmetic errors
 - b) Code redundancy and syntax errors

- c) Inefficient memory usage and slow program execution
 - d) Lack of support for dynamic memory allocation
11. In the provided example, what is the purpose of the function swapNumbers?
- a) It swaps two variables using call by reference.
 - b) It increments a variable using call by value.
 - c) It checks if two variables are equal.
 - d) It calculates the factorial of a number.
12. What is the main difference between "call by value" and "call by reference"?
- a) Call by value uses pointers, while call by reference does not.
 - b) Call by reference works with copies of variables.
 - c) Call by value directly modifies the original variable.
 - d) Call by reference passes the memory address of the variable.
13. What is the primary advantage of using smart pointers in modern C++?
- a) They facilitate pointer arithmetic.
 - b) They prevent memory leaks and provide safer memory management.
 - c) They allow dynamic memory allocation without the need for pointers.
 - d) They improve the performance of pointer operations.
14. What is the purpose of the new and delete keywords in C++ dynamic memory allocation?
- a) They declare new variables.
 - b) They allocate and deallocate memory at runtime.
 - c) They perform mathematical operations on pointers.
 - d) They are reserved keywords for pointer functions

8.11 REFERENCES

- "The C++ Programming Language" by Bjarne Stroustrup:
 - Author: Bjarne Stroustrup (the creator of C++)
 - Description: This is considered the definitive book for learning C++. It covers the language in-depth and provides insights into its design and evolution.
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo:
 - Authors: Stanley B. Lippman, Josée Lajoie, Barbara E. Moo
 - Description: Known for its clear and concise style, this book is suitable for beginners and serves as an excellent reference for more advanced topics.

- "Effective C++: 55 Specific Ways to Improve Your Programs and Designs" by Scott Meyers:
 - Author: Scott Meyers
 - Description: This book provides practical advice and tips for writing better C++ code. It focuses on best practices and common pitfalls.
- "Accelerated C++: Practical Programming by Example" by Andrew Koenig and Barbara E. Moo:
 - Authors: Andrew Koenig, Barbara E. Moo
 - Description: Aimed at those new to programming, this book takes a practical approach to learning C++. It emphasizes writing real programs early in the learning process.
- "C++ Standard Library - A Tutorial and Reference" by Nicolai M. Josuttis:
 - Author: Nicolai M. Josuttis
 - Description: This book provides a comprehensive guide to the C++ Standard Library. It covers the containers, algorithms, and other components of the library, offering both tutorials and reference material.

UNIT 9: OBJECTS AND CLASSES

CONTENT:

▪ *Objectives*

9.1 Introduction

9.2 Class

9.3 Object

9.4 Defining the Class

9.5 Private and Public

9.6 Constructors

9.7 Destructors

9.8 Summary

9.9 Keywords

9.10 Exercise

9.11 References

OBJECTIVES:

After studying this unit, you will be able to:

- Describe the fundamental concepts of classes and objects in object-oriented programming (OOP).
- Explain the role of a class as a blueprint or template for creating objects, defining attributes and behaviors.
- Understand the concept of an object as an instance or realization of a class with unique attributes and behaviors.
- Highlight the benefits of using classes and objects in organizing code, making it modular, and facilitating work with complex systems.

9.1 INTRODUCTION

Now, we're going to talk about something you've been waiting for: objects and classes. We've already covered some basics. We've learned about structures, which help organize data. We've also looked at functions, which are like named actions in a program. In this chapter, we'll combine these concepts to make classes. We'll start with simple classes and move on to more complex ones. We'll begin by looking at the details of classes and objects. Towards the end of the chapter, we'll talk about the benefits of using the object-oriented programming approach.

Classes and objects are fundamental concepts in object-oriented programming (OOP).

9.2 CLASS

A class is like a blueprint or template for creating objects. It defines the attributes (data) and behaviors (functions or methods) that objects of that class will have. Think of it as a description of what an object should be like. For example, if you have a "Car" class, it might define the car's properties like colour, make, and model, as well as methods like "start" and "stop."

9.3 OBJECT

An object is an instance or a specific realization of a class. It is created based on the class's blueprint. When you create an object from a class, you are essentially creating a unique version of that class with its own set of attributes and the ability to perform actions defined in the class. For example, you can create specific car objects, like "MyCar" and "YourCar," based on the "Car" class, each with its own unique attributes and behaviour.

In simpler terms, a class is like a plan for making something, and an object is the actual thing made according to that plan. Classes and objects help organize code, make it more modular, and enable you to work with complex systems by breaking them down into manageable parts.

Let's use a common real-world example to illustrate classes and objects: a "Person" class and instances (objects) of that class.

9.4 DEFINING THE CLASS

Here's the definition (sometimes called a specifier) for the class `smallobj`, copied from the

SMALLOBJ listing:

```
class smallobj //define a class

{

private:

int somedata; //class data

public:

void setdata(int d) //member function to set data

{ somedata = d; }

void showdata() //member function to display data

{ cout << "\nData is " << somedata; }

};
```

The definition starts with the keyword `class`, followed by the class name—`smallobj` in this example. Like a structure, the body of the class is delimited by braces and terminated by a semicolon. (Don't forget the semicolon. Remember, data constructs such as structures and classes end with a semicolon, while control constructs such as functions and loops do not.)

9.5 PRIVATE AND PUBLIC

A key feature of object-oriented programming is data hiding. This term does not refer to the activities of particularly paranoid programmers; rather it means that data is concealed within a class so that it cannot be accessed mistakenly by functions outside the class. The primary mechanism for hiding data is to put it in a class and make it private. Private data or functions can only be accessed from within the class. Public data or functions, on the other hand, are accessible from outside the class.

Data hiding, on the other hand, means hiding data from parts of the program that don't need to access it. More specifically, one class's data is hidden from other classes. Data hiding is designed to protect well-intentioned programmers from honest mistakes.

Usually the data within a class is private and the functions are public. This is a result of the way classes are used. The data is hidden so it will be safe from accidental manipulation, while the functions that operate on the data are public so they can be accessed from outside the class. However, there is no rule that says data must be private and functions public; in some circumstances you may find you'll need to use private functions and public data.

Class Definition (Person Class):

```
#include <iostream>

using namespace std;

class Person {

public:

    // Constructor to initialize the object

    Person(string name, int age) {

        this->name = name;

        this->age = age;

    }

    // Member function to display information

    void introduce() {

        cout << "Hello, my name is " << name << " and I am " << age << " years old." << endl;

    }

private:

    string name;

    int age;

};
```

```
int main() {  
  
    // Create objects of the Person class  
  
    Person person1("Alice", 30);  
  
    Person person2("Bob", 25);  
  
    // Call member functions on objects  
  
    person1.introduce();  
  
    person2.introduce();  
  
    return 0;  
  
}
```

In this C++ example:

1. We define a "Person" class with a constructor to initialize the object's attributes (name and age) and a member function introduce to display information about the person.
2. In the main function, we create two objects, person1 and person2, of the "Person" class.
3. We then call the introduce member function on each object to display personalized information about the persons.

9.6 CONSTRUCTORS

In C++, a constructor is a special member function within a class that is automatically called when an object of the class is created. Constructors are used to initialize the object's data members or perform any necessary setup tasks. Here are the key points about constructors in C++:

1. **Initialization:** Constructors are primarily used for initializing the data members (attributes) of a class when an object is created. They ensure that the object starts with valid and appropriate initial values.

2. Same Name as Class: Constructors have the same name as the class and do not have a return type, not even "void." This is a key feature that distinguishes them from regular member functions.
3. Automatic Invocation: Constructors are called automatically when an object of the class is created. You don't need to explicitly call them; they are invoked as part of the object creation process.
4. Default Constructor: If you don't provide any constructors in your class, C++ provides a default constructor with no parameters. It initializes the data members to default values.
5. Parameterized Constructors: You can define constructors with parameters to allow different ways of initializing objects. These parameters can be used to set the initial values of data members based on arguments provided during object creation.

Here's an example of a class with constructors in C++:

```
#include <iostream>

using namespace std;

class MyClass {

public:

    // Default constructor

    MyClass() {

        cout << "Default constructor called." << endl;

        value = 0;

    }

    // Parameterized constructor

    MyClass(int val) {

        cout << "Parameterized constructor called." << endl;
```

```
        value = val;

    }

    void display() {

        cout << "Value: " << value << endl;

    }

private:

    int value;

};

int main() {

    MyClass obj1;        // Calls the default constructor

    MyClass obj2(42);    // Calls the parameterized constructor

    obj1.display();      // Displays: Value: 0

    obj2.display();      // Displays: Value: 42

    return 0;

}
```

9.7 DESTRUCTORS

We've seen that a special member function—the constructor—is called automatically when an object is first created. You might guess that another function is called automatically when an object is destroyed. This is indeed the case. Such a function is called a destructor. A destructor has the same name as the constructor (which is the same as the class name) but is preceded by a tilde:

```
class Foo

{
```



```
private:

int data;

public:

Foo() : data(0) //constructor (same name as class)

{ }

~Foo() //destructor (same name with tilde)

{ }

};
```

Like constructors, destructors do not have a return value. They also take no arguments (the assumption being that there's only one way to destroy an object).

The most common use of destructors is to deallocate memory that was allocated for the object by the constructor.

9.8 SUMMARY

The study on object-oriented programming introduces the fundamental concepts of classes and objects. In this context, a class serves as a blueprint or template for creating objects, defining their attributes and behaviors. Objects, in turn, are instances of these classes, possessing unique characteristics and the ability to perform actions outlined by the class. The importance of classes and objects lies in their role in organizing code, making it modular, and simplifying the management of complex systems by breaking them down into manageable parts.

The practical application of these concepts is exemplified through a C++ "Person" class. This class features a constructor for initializing object attributes, member functions for specific actions, and private data members to implement data hiding. The significance of data hiding is underscored, ensuring that data is protected from unintentional access outside the class. Furthermore, the study delves into the crucial aspects of constructors in C++, highlighting their automatic invocation during object creation, role in initializing data members, and the

distinction from regular member functions. Overall, the unit equips learners with essential knowledge for effective object-oriented programming.

9.9 KEYWORDS

Object-oriented programming, Classes, Objects, Blueprint, Template, Attributes, Modular code, Constructor, Member functions, Private data members, Data protection, Initialization, Destructors

9.10 EXERCISE

1. What is the fundamental concept of a class in object-oriented programming (OOP)?
2. How does a class serve as a blueprint or template for creating objects?
3. Define the term "object" and explain its role in OOP.
4. What are the attributes and behaviors of a class, and how are they defined?
5. Explain the concept of data hiding and its significance in OOP.
6. How do classes and objects contribute to organizing code and making it modular?
7. Provide an example of a real-world class and instances (objects) based on it.
8. In C++, what is the purpose of a constructor, and how is it different from a regular member function?
9. How are private data members used in a class, and why is data protection important?
10. What is the significance of destructors in C++, and what tasks do they typically perform?
11. How does the use of classes and objects enhance the manageability of complex systems in programming?
12. Can you explain the process of creating objects from a class using the given C++ "Person" class example?
13. Why is it essential to understand the distinction between private and public in the context of classes?
14. Discuss the benefits of using object-oriented programming in code development.
15. How do constructors contribute to ensuring that objects start with valid and appropriate initial values?

9.11 REFERENCES

- "The C++ Programming Language" by Bjarne Stroustrup:
 - Author: Bjarne Stroustrup (the creator of C++)

- Description: This is considered the definitive book for learning C++. It covers the language in-depth and provides insights into its design and evolution.
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo:
 - Authors: Stanley B. Lippman, Josée Lajoie, Barbara E. Moo
 - Description: Known for its clear and concise style, this book is suitable for beginners and serves as an excellent reference for more advanced topics.
- "Effective C++: 55 Specific Ways to Improve Your Programs and Designs" by Scott Meyers:
 - Author: Scott Meyers
 - Description: This book provides practical advice and tips for writing better C++ code. It focuses on best practices and common pitfalls.
- "Accelerated C++: Practical Programming by Example" by Andrew Koenig and Barbara E. Moo:
 - Authors: Andrew Koenig, Barbara E. Moo
 - Description: Aimed at those new to programming, this book takes a practical approach to learning C++. It emphasizes writing real programs early in the learning process.
- "C++ Standard Library - A Tutorial and Reference" by Nicolai M. Josuttis:
 - Author: Nicolai M. Josuttis

Description: This book provides a comprehensive guide to the C++ Standard Library. It covers the containers, algorithms, and other components of the library, offering both tutorials and reference material.

UNIT 10: OPERATOR OVERLOADING

CONTENT:

▪ Objectives

10.1 Introduction

10.2 Overloading Unary Operators

10.3 Overloading Arithmetic Operators

10.4 Summary

10.5 Keywords

10.6 Exercise

10.7 References

OBJECTIVES:

After studying this unit, you will be able to:

- Gain a clear understanding of operator overloading in C++ to redefine existing operators and create new ones for user-defined data types or classes.
- Learn to implement operator overloading functions—special member functions named after operators—to customize how operators work with class objects.
- Explore practical applications of customizing operators, focusing on arithmetic, comparison, and assignment operators.
- Explore the customization of unary and arithmetic operators, including a "Complex" class adding complex numbers and a "Distance" class handling distance addition for real-world insights.

10.1 INTRODUCTION

Operator overloading in C++ is a feature that allows you to define and customize the behavior of operators for user-defined data types or classes. C++ allows you to redefine the meaning of existing operators or create new operators for your custom classes. This is achieved by providing special member functions for operators, which are called operator overloading functions.

Here are some key points about operator overloading in C++:

- **Operator Overloading Functions:** To overload an operator for a class, you define a special member function that has the operator symbol as its name. For example, to overload the addition operator `+`, you define a function named `operator+`.
- **Customized Behavior:** You can customize how operators work with objects of your class. For instance, you can define how the `+` operator should add two objects or how the `==` operator should compare them.
- **Infix Notation:** Operator overloading allows you to use the overloaded operators in an infix notation, like how built-in operators work. This makes the code more intuitive and readable.
- **Overloaded Operators:** Many operators can be overloaded, including arithmetic operators (`+`, `-`, `*`, `/`, `%`), comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`), assignment operators (`=`, `+=`, `-=`, `*=`, `/=`), and more.

10.2 OVERLOADING UNARY OPERATORS

Here's a simple example of operator overloading in C++ where we define an `operator+` function for a custom "Complex" class to add two complex numbers:

Let's start off by overloading a unary operator. "Objects and Classes," we created a class `Counter` to keep track of a count. Objects of that class were incremented by calling a member function:

```
c1.inc_count();
```

That did the job, but the listing would have been more readable if we could have used the increment operator `++` instead:

```
++c1;
```

All dyed-in-the-wool C++ (and C) programmers would guess immediately that this expression increments `c1`.

Let's rewrite `COUNTER` to make this possible. Here's the listing for `COUNTPP1`:

```
#include <iostream>

using namespace std;

class Counter

{

private:

    unsigned int count; //count

public:

    Counter() : count(0) //constructor

    { }

    unsigned int get_count() //return count

    { return count; }

    void operator ++ () //increment (prefix)

    {

        ++count;

    }

};

int main()
```

```
{  
  
Counter c1, c2; //define and initialize  
  
cout << "\nc1=" << c1.get_count(); //display  
  
cout << "\nc2=" << c2.get_count();  
  
++c1; //increment c1  
  
++c2; //increment c2  
  
++c2; //increment c2  
  
cout << "\nc1=" << c1.get_count(); //display again  
  
cout << "\nc2=" << c2.get_count() << endl;  
  
return 0;  
  
}
```

In this program we create two objects of class Counter: c1 and c2. The counts in the objects are displayed; they are initially 0. Then, using the overloaded ++ operator, we increment c1 once and c2 twice and display the resulting values.

How do we teach a normal C++ operator to act on a user-defined operand? The keyword operator is used to overload the ++ operator in this declarator:

```
void operator ++ ()
```

The return type (void in this case) comes first, followed by the keyword operator, followed by the operator itself (++), and finally the argument list enclosed in parentheses (which are empty here). This declarator syntax tells the compiler to call this member function whenever the ++ operator is encountered, provided the operand (the variable operated on by the ++) is of type Counter.

10.3 OVERLOADING ARITHMETIC OPERATORS

In the program we showed how two English Distance objects could be added using overloading the + operator. We can reduce this dense-looking expression to

```
dist3 = dist1 + dist2;
```

```
#include <iostream>
```

```
using namespace std;
```

```
class Distance //English Distance class
```

```
{
```

```
private:
```

```
int feet;
```

```
float inches;
```

```
public: //constructor (no args)
```

```
Distance() : feet(0), inches(0.0)
```

```
{ } //constructor (two args)
```

```
Distance(int ft, float in) : feet(ft), inches(in)
```

```
{ }
```

```
void getdist() //get length from user
```

```
{
```

```
cout << "\nEnter feet: "; cin >> feet;
```

```
cout << "Enter inches: "; cin >> inches;
```

```
}
```

```
void showdist() const //display distance
```



```
{ cout << feet << "\'-" << inches << '\''; }
```

```
Distance operator + ( Distance ) const; //add 2 distances
```

```
};
```

```
//add this distance to d2
```

```
Distance Distance::operator + (Distance d2) const //return sum
```

```
{
```

```
int f = feet + d2.feet; //add the feet
```

```
float i = inches + d2.inches; //add the inches
```

```
if(i >= 12.0) //if total exceeds 12.0,
```

```
{ //then decrease inches
```

```
i -= 12.0; //by 12.0 and
```

```
f++; //increase feet by 1
```

```
} //return a temporary Distance
```

```
return Distance(f,i); //initialized to sum
```

```
}
```

```
int main()
```

```
{
```

```
Distance dist1, dist3, dist4; //define distances
```

```
dist1.getdist(); //get dist1 from user
```

```
Distance dist2(11, 6.25); //define, initialize dist2
```

```
dist3 = dist1 + dist2; //single '+' operator
```

```

dist4 = dist1 + dist2 + dist3; //multiple '+' operators

//display all lengths

cout << "dist1 = "; dist1.showdist(); cout << endl;

cout << "dist2 = "; dist2.showdist(); cout << endl;

cout << "dist3 = "; dist3.showdist(); cout << endl;

cout << "dist4 = "; dist4.showdist(); cout << endl;

return 0;

}

```

To show that the result of an addition can be used in another addition as well as in an assignment, another addition is performed in main(). We add dist1, dist2, and dist3 to obtain dist4 (which should be double the value of dist3), in the statement.

```
dist4 = dist1 + dist2 + dist3;
```

Here's the output from the program:

Enter feet: 10

Enter inches: 6.5

dist1 = 10'-6.5" ←from user

dist2 = 11'-6.25" ←initialized in program

dist3 = 22'-0.75" ←dist1+dist2

dist4 = 44'-1.5" ← dist1+dist2+dist3

10.4 SUMMARY

The chapter on Operator Overloading in C++ introduces a crucial feature that empowers developers to redefine and customize the behavior of operators for user-defined classes. By implementing special member functions, known as operator overloading functions,

programmers can achieve a higher level of control over how operators interact with objects of their classes. The content emphasizes the significance of customized behavior for operators and elucidates the process of overloading various operators, including arithmetic, comparison, and assignment operators.

The practical application of operator overloading is exemplified through code snippets. For instance, a "Complex" class is used to add two complex numbers, showcasing how operators like '+' can be tailored for specific class behaviors. Additionally, the content presents a real-world scenario involving a "Distance" class, demonstrating the overloading of the '+' operator to handle the addition of distances. The provided examples, such as overloading unary operators and arithmetic operators, contribute to a comprehensive understanding of operator overloading in C++. Overall, the content serves as a valuable resource for programmers seeking to enhance the functionality of their classes through effective operator overloading techniques.

10.5 KEYWORDS

Operator Overloading, User-defined classes, Special member functions, Customized behavior, Arithmetic operators, Comparison operators, Assignment operators

10.6 EXERCISE

1. What is the primary focus of the chapter on Operator Overloading in C++?
2. How does operator overloading empower developers in customizing the behavior of operators for user-defined classes?
3. What are the special member functions used for operator overloading called?
4. Why is customized behavior for operators significant in programming with user-defined classes?
5. Can you provide examples of operators that can be overloaded, as mentioned in the content?
6. How is the process of overloading arithmetic operators explained in the chapter?
7. What is the practical application of operator overloading demonstrated through the "Complex" class example?
8. In what way does the content showcase the overloading of the '+' operator for a "Distance" class dealing with distances?
9. What role do code snippets play in understanding operator overloading in C++?

10. How do examples of overloading unary operators contribute to a comprehensive understanding of the concept in the provided content?

10.7 REFERENCES

- "The C++ Programming Language" by Bjarne Stroustrup:
 - Author: Bjarne Stroustrup (the creator of C++)
 - Description: This is considered the definitive book for learning C++. It covers the language in-depth and provides insights into its design and evolution.
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo:
 - Authors: Stanley B. Lippman, Josée Lajoie, Barbara E. Moo
 - Description: Known for its clear and concise style, this book is suitable for beginners and serves as an excellent reference for more advanced topics.
- "Effective C++: 55 Specific Ways to Improve Your Programs and Designs" by Scott Meyers:
 - Author: Scott Meyers
 - Description: This book provides practical advice and tips for writing better C++ code. It focuses on best practices and common pitfalls.
- "Accelerated C++: Practical Programming by Example" by Andrew Koenig and Barbara E. Moo:
 - Authors: Andrew Koenig, Barbara E. Moo
 - Description: Aimed at those new to programming, this book takes a practical approach to learning C++. It emphasizes writing real programs early in the learning process.
- "C++ Standard Library - A Tutorial and Reference" by Nicolai M. Josuttis:
 - Author: Nicolai M. Josuttis
 - Description: This book provides a comprehensive guide to the C++ Standard Library. It covers the containers, algorithms, and other components of the library, offering both tutorials and reference material.

UNIT 11: INHERITANCE

CONTENT:

▪ Objectives

11.1 Introduction

11.2 The protected Access Specifier

11.3 Types of Inheritance

11.4 Summary

11.5 Keywords

11.6 Exercise

11.7 References

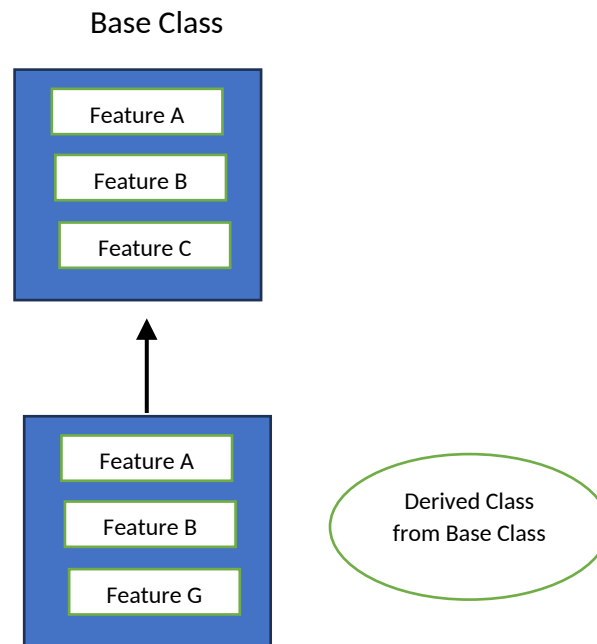
OBJECTIVES:

After studying this unit, you will be able to:

- Understanding Virtual Functions: Grasp the concept of virtual functions, enabling dynamic function execution based on content.
- Implementing Virtual Functions: Learn to apply the 'virtual' keyword to enable dynamic binding and polymorphic behavior in C++.
- Late Binding and Dynamic Binding: Comprehend the compiler's deferred decision-making during runtime, enhancing flexibility through late binding.
- Pure Virtual Functions: Master the use of pure virtual functions to create abstract classes, restricting object instantiation.

11.1 INTRODUCTION

Inheritance is probably the most powerful feature of object-oriented programming, after classes themselves. Inheritance is the process of creating new classes, called derived classes, from existing or base classes. The derived class inherits all the capabilities of the base class but can add embellishments and refinements of its own. The base class is unchanged by this process. The inheritance relationship is shown in Figure:



Inheritance is a fundamental concept in object-oriented programming (OOP) and is a key feature of C++. It allows you to create a new class (derived or child class) based on an existing class (base or parent class). Inheritance enables the child class to inherit the properties and behaviors (data members and member functions) of the parent class. The child class can then extend or modify these inherited properties and behaviours and add its own.

Here are some important points about inheritance in C++:

- **Base Class (Parent Class):** The class from which another class inherits is called the base class or parent class. It defines the common attributes and behaviors that will be shared by derived classes.

- **Derived Class (Child Class):** The class that inherits from a base class is called the derived class or child class. It can add additional attributes and behaviors to the ones inherited from the base class.
- **"is-a" Relationship:** Inheritance models an "is-a" relationship, where the child class is a specialized version of the parent class. For example, if you have a base class "Animal," you can derive child classes like "Dog" and "Cat."
- **Access Control:** In C++, you can specify access control for the inherited members. By default, public members of the base class become public members in the derived class, while private members of the base class remain inaccessible in the derived class.
- **Syntax:** In C++, you indicate inheritance by using the class keyword, followed by a colon and the access specifier (public, protected, or private), and then the base class name. For example:

```
class DerivedClass : access-specifier BaseClass {  
  
    // Additional members of the derived class  
  
};
```

Here is an Example:

```
#include <iostream>  
  
using namespace std;  
  
// Base class  
  
class Animal {  
  
public:  
  
    void eat() {  
  
        cout << "Animal is eating." << endl;  
  
    }  
  
};
```

```
// Derived class

class Dog : public Animal {

public:

    void bark() {

        cout << "Dog is barking." << endl;

    }

};

int main() {

    Dog myDog;

    myDog.eat(); // Inherited from the base class

    myDog.bark(); // Defined in the derived class

    return 0;

}
```

In this example, the "Dog" class inherits from the "Animal" class. The "Dog" class inherits the eat method from the base class and has its own method, bark. Inheritance allows us to reuse and extend the functionality defined in the base class, promoting code reuse, and maintaining a hierarchical structure in object-oriented programs.

11.2 THE PROTECTED ACCESS SPECIFIER

We have increased the functionality of a class without modifying it. Well, almost without modifying it. Let's look at the single change we made to the Counter class.

The moral is that if you are writing a class that you suspect might be used, at any point in the future, as a base class for other classes, then any member data that the derived classes might need to access should be made protected rather than private. This ensures that the class is "inheritance ready."

11.3 TYPES OF INHERITANCE

Inheritance is a key concept in object-oriented programming (OOP) that allows a class (called the subclass or derived class) to inherit properties and behaviors from another class (called the superclass or base class). There are several types of inheritance, each serving specific purposes in software design. The main types of inheritance are:

11.3.1 Single Inheritance:

Definition: In single inheritance, a class inherits from only one base class.

Example: Class B inherits from Class A.

```
class A {  
  
    // Class A members and methods  
  
};  
  
class B : public A {  
  
    // Class B members and methods  
  
};
```

11.3.2 Multiple Inheritance:

Definition: In multiple inheritance, a class can inherit from more than one base class. This allows the derived class to have characteristics of multiple classes.

Example: Class C inherits from both Class A and Class B.

```
class A {  
  
    // Class A members and methods  
  
};  
  
class B : public A {
```

```
// Class B members and methods  
  
};
```

11.3.3 Multilevel Inheritance:

Definition: In multilevel inheritance, a class inherits from another class, and then another class inherits from the derived class.

Example: Class C inherits from Class B, which in turn inherits from Class A.

```
class A {  
  
    // Class A members and methods  
  
};  
  
class B : public A {  
  
    // Class B members and methods  
  
};  
  
class C : public B {  
  
    // Class C members and methods  
  
};
```

11.3.4 Hierarchical Inheritance:

Definition: In hierarchical inheritance, multiple classes inherit from a single base class.

Example: Classes B and C inherit from Class A.

```
class A {  
  
    // Class A members and methods
```

```
};

class B : public A {

    // Class B members and methods

};

class C : public A {

    // Class C members and methods

};
```

11.3.5 Hybrid (or Virtual) Inheritance:

Definition: Hybrid inheritance is a combination of any two or more types of inheritance mentioned above. It often involves the use of multiple and multilevel inheritance together.

Example: A class D inherits from both classes B and C, where B and C may be inherited from class A.

```
class A {

    // Class A members and methods

};

class B : public A {

    // Class B members and methods

};

class C : public A {

    // Class C members and methods

};
```

```
class D : public B, public C {  
  
    // Class D members and methods  
  
};
```

It's important to note that while inheritance is a powerful tool in OOP, it should be used judiciously to promote code reusability and maintainability. Excessive or improper use of inheritance can lead to design complexities and maintenance challenges.

11.4 SUMMARY

The chapter introduces virtual functions in C++, providing a metaphorical understanding: virtual, like an illusion, enables a program to execute functions that appear to belong to one object type but may work differently. Virtual functions allow uniform treatment of diverse objects, promoting program flexibility. The concept is exemplified using a graphics program scenario with shapes, showcasing the practicality of using one command for various objects, making the program adaptable.

Two critical scenarios are explored: normal member functions accessed with pointers and virtual member functions accessed with pointers. The former illustrates the compiler choosing functions based on pointer type, while the latter, using 'virtual,' triggers specific functions based on pointer content, a feature known as polymorphism. Late binding or dynamic binding further enhances this by deferring function selection until runtime, providing increased power and flexibility.

The chapter also introduces pure virtual functions, enforcing abstract classes by preventing object instantiation of the base class. The use of "=0" in a function declaration signals pure virtual status. The implementation of pure virtual functions in derived classes is mandatory, emphasizing consistency for creating objects. The program's output demonstrates the effectiveness of pure virtual functions in restricting object instantiation, providing a clear distinction between abstract and concrete classes.

11.5 KEYWORDS

Virtual Functions, Polymorphism, Late Binding, Dynamic Binding, Pure Virtual Function Abstract Class, Function Overriding, C++ Programming

11.6 EXERCISE

1. What is the purpose of virtual functions in C++?
2. How does polymorphism contribute to program flexibility in C++?
3. Explain the difference between normal and virtual member functions accessed with pointers.
4. What is late binding, and how does it enhance the power of virtual functions?
5. Why is the keyword 'virtual' used in the declaration of a virtual function?
6. How does the use of pure virtual functions lead to the creation of abstract classes?
7. Why does the program output change when the 'virtual' keyword is added to the base class function?
8. What happens if you attempt to create an object of an abstract class in C++?
9. In what scenario might you choose to make all virtual functions in a base class pure?
10. How does dynamic binding differ from static binding in C++?

11.7 REFERENCES

- "The C++ Programming Language" by Bjarne Stroustrup:
 - Author: Bjarne Stroustrup (the creator of C++)
 - Description: This is considered the definitive book for learning C++. It covers the language in-depth and provides insights into its design and evolution.
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo:
 - Authors: Stanley B. Lippman, Josée Lajoie, Barbara E. Moo
 - Description: Known for its clear and concise style, this book is suitable for beginners and serves as an excellent reference for more advanced topics.
- "Effective C++: 55 Specific Ways to Improve Your Programs and Designs" by Scott Meyers:
 - Author: Scott Meyers
 - Description: This book provides practical advice and tips for writing better C++ code. It focuses on best practices and common pitfalls.
- "Accelerated C++: Practical Programming by Example" by Andrew Koenig and Barbara E. Moo:
 - Authors: Andrew Koenig, Barbara E. Moo

- Description: Aimed at those new to programming, this book takes a practical approach to learning C++. It emphasizes writing real programs early in the learning process.
- "C++ Standard Library - A Tutorial and Reference" by Nicolai M. Josuttis:
 - Author: Nicolai M. Josuttis
 - Description: This book provides a comprehensive guide to the C++ Standard Library. It covers the containers, algorithms, and other components of the library, offering both tutorials and reference material.

UNIT 12: VIRTUAL FUNCTIONS

CONTENT:

▪ **Objectives**

12.1 Introduction

12.2 Normal Member Functions Accessed with Pointers

12.3 Virtual Member Functions Accessed with Pointers

12.4 Late Binding

12.5 Pure Virtual Function

12.6 Summary

12.7 Keywords

12.8 Exercise

12.9 References

OBJECTIVES:

After studying this unit, you will be able to:

- Grasp the concept of virtual functions, enabling dynamic function execution based on content.
- Learn to apply the 'virtual' keyword to enable dynamic binding and polymorphic behavior in C++.
- Comprehend the compiler's deferred decision-making during runtime, enhancing flexibility through late binding.
- Master the use of pure virtual functions to create abstract classes.

12.1 INTRODUCTION

Virtual means something that seems real but isn't exactly as it appears. When we talk about virtual functions, it's like using a trick in a program. Imagine you have many different objects, like shapes in a graphics program—triangles, balls, squares, etc. Each shape has a special function called `draw()` that makes it appear on the screen.

Now, let's say you want to put all these shapes into a list and use the same command to make them all appear on the screen. This is where virtual functions come in handy. They allow a program to use a function that looks like it belongs to one type of object but, in reality, it might be doing the work of a different type of object.

In simpler terms, virtual functions help us treat different objects in a similar way, even if they belong to different classes. It's like a neat trick that lets us use one command for various objects, making our program more flexible.

This is a remarkable feature: The same function call can make completely different actions happen. If the pointer in `ptrarr` is pointing to a ball, it triggers the function that draws a ball. If it points to a triangle, it activates the function that draws a triangle. This phenomenon is known as polymorphism, which essentially means having different forms. Despite looking the same with the `draw()` expression, these functions perform differently based on what's inside `ptrarr[j]`. Polymorphism is a crucial aspect of object-oriented programming, alongside classes and inheritance.

12.2 NORMAL MEMBER FUNCTIONS ACCESSED WITH POINTERS

In our initial example, we demonstrate the outcome when both a base class and its derived classes contain functions sharing the same name. The access to these functions is done through pointers, but without employing virtual functions.

```
#include <iostream>
```

```
using namespace std;
```

```
class Base //base class
```

```
{
```



```
public:

void show() //normal function

{ cout << "Base\n"; }

};

class Derv1 : public Base //derived class 1

{

public:

void show()

{ cout << "Derv1\n"; }

};

class Derv2 : public Base //derived class 2

{

public:

void show()

{ cout << "Derv2\n"; }

};

int main()

{

Derv1 dv1;    //object of derived class 1

Derv2 dv2;    //object of derived class 2

Base* ptr;    //pointer to base class
```

```

ptr = &dv1;    //put address of dv1 in pointer

ptr->show();    //execute show()

ptr = &dv2;    //put address of dv2 in pointer

ptr->show();    //execute show()

return 0;

}

```

The output from the program answers these questions:

Base

Base

As you can see, the function in the base class is always executed. The compiler ignores the contents of the pointer ptr and chooses the member function that matches the type of the pointer.

12.3 VIRTUAL MEMBER FUNCTIONS ACCESSED WITH POINTERS

Let's modify our program with a single change: We'll add the keyword "virtual" before the declaration of the show() function in the base class.

```

#include <iostream>

using namespace std;

class Base    //base class

{

public:

virtual void show()    //virtual function

{ cout << "Base\n"; }

```

```
};

class Derv1 : public Base    //derived class 1

{

public:

void show()

{ cout << "Derv1\n"; }

};

class Derv2 : public Base    //derived class 2

{

public:

void show()

{ cout << "Derv2\n"; }

};

int main()

{

Derv1 dv1;    //object of derived class 1

Derv2 dv2;    //object of derived class 2

Base* ptr;    //pointer to base class

ptr = &dv1;    //put address of dv1 in pointer

ptr->show();    //execute show()

ptr = &dv2;    //put address of dv2 in pointer
```

```
ptr->show(); //execute show()

return 0;

}
```

The output of this program is:

Derv1

Derv2

Now, as observed, it's the member functions of the derived classes, not the base class, that get executed. When we alter the content of `ptr` from pointing to `Derv1` to `Derv2`, the specific instance of `show()` being executed also changes. Therefore, the same function call `ptr->show()`; triggers different functions, depending on the content of `ptr`. The rule here is that the compiler chooses the function based on the content of the pointer `ptr`, not its type.

12.4 LATE BINDING

The discerning reader might wonder how the compiler determines which function to compile. In the above program scenario, the compiler faces no issue with the expression `ptr->show()`. It consistently compiles a call to the `show()` function in the base class. However, in the VIRT scenario, the compiler is uncertain about the class the contents of `ptr` may belong to. It could be pointing to an object of the `Derv1` class or the `Derv2` class. Which version of `draw()` should the compiler invoke? In reality, the compiler is unsure, so it defers the decision until the program is in execution. During runtime, when it's known which class is being pointed to by `ptr`, the appropriate version of `draw()` will be called. This is referred to as late binding or dynamic binding. (The conventional method of selecting functions during compilation is known as early binding or static binding.) Late binding involves some additional overhead but provides enhanced power and flexibility.

12.5 PURE VIRTUAL FUNCTION

How do we communicate to users of our class family that we don't want them to create objects of the base class? While we could rely on documentation, it's more effective to structure our classes in a way that makes such instantiation impossible. How can we achieve this? By

including at least one pure virtual function in the base class. A pure virtual function is identified by adding the expression "=0" to its declaration.

```
#include <iostream>

using namespace std;

class Base    //base class

{

public:

virtual void show() = 0;    //pure virtual function

};

class Derv1 : public Base    //derived class 1

{

public:

void show()

{ cout << "Derv1\n"; }

};

class Derv2 : public Base    //derived class 2

{

public:

void show()

{ cout << "Derv2\n"; }

};
```

```
int main()

{

// Base bad;           //can't make object from abstract class

Base* arr[2];          //array of pointers to base class

Derv1 dv1;             //object of derived class 1

Derv2 dv2;             //object of derived class 2

arr[0] = &dv1;          //put address of dv1 in array

arr[1] = &dv2;          //put address of dv2 in array

arr[0]->show();         //execute show() in both objects

arr[1]->show();

return 0;

}
```

Here the virtual function show() is declared as

```
virtual void show() = 0;           // pure virtual function
```

The equal sign used here doesn't involve assignment; it doesn't assign the value 0 to anything. The "=0" syntax is simply a way to inform the compiler that a virtual function is intended to be pure. If you attempt to create objects of the Base class in the main() function, the compiler will raise an error, indicating an attempt to instantiate an object of an abstract class. The error message will also specify the name of the pure virtual function responsible for making it an abstract class.

It's worth noting that, even though this is just a declaration, you don't need to provide a definition for the base class show(). However, you have the option to do so if necessary. Once you introduce a pure virtual function in the base class, you must override it in all derived classes from which you intend to create objects. If a class fails to override the pure virtual function, it

becomes an abstract class itself, and you cannot instantiate objects from it (though you may do so from classes derived from it). For consistency, you might choose to make all virtual functions in the base class pure.

In the above scenario, you'll notice another unrelated change: the addresses of the member functions are stored in an array of pointers and accessed using array elements. This functions in the same way as using a single pointer.

The output of Program is the same as Virtual function:

Derv1

Derv2

12.6 SUMMARY

The chapter introduces virtual functions in C++, providing a metaphorical understanding: virtual, like an illusion, enables a program to execute functions that appear to belong to one object type but may work differently. Virtual functions allow uniform treatment of diverse objects, promoting program flexibility. The concept is exemplified using a graphics program scenario with shapes, showcasing the practicality of using one command for various objects, making the program adaptable.

Two critical scenarios are explored: normal member functions accessed with pointers and virtual member functions accessed with pointers. The former illustrates the compiler choosing functions based on pointer type, while the latter, using 'virtual,' triggers specific functions based on pointer content, a feature known as polymorphism. Late binding or dynamic binding further enhances this by deferring function selection until runtime, providing increased power and flexibility.

The chapter also introduces pure virtual functions, enforcing abstract classes by preventing object instantiation of the base class. The use of "=0" in a function declaration signals pure virtual status. The implementation of pure virtual functions in derived classes is mandatory, emphasizing consistency for creating objects. The program's output demonstrates the

effectiveness of pure virtual functions in restricting object instantiation, providing a clear distinction between abstract and concrete classes.

12.7 KEYWORDS

Virtual Functions, Polymorphism, Late Binding, Dynamic Binding, Pure Virtual Function, Abstract Class, Function Overriding, C++ Programming

12.8 EXERCISE

1. What is the purpose of virtual functions in C++?
2. How does polymorphism contribute to program flexibility in C++?
3. Explain the difference between normal and virtual member functions accessed with pointers.
4. What is late binding, and how does it enhance the power of virtual functions?
5. Why is the keyword 'virtual' used in the declaration of a virtual function?
6. How does the use of pure virtual functions lead to the creation of abstract classes?
7. Why does the program output change when the 'virtual' keyword is added to the base class function?
8. What happens if you attempt to create an object of an abstract class in C++?
9. In what scenario might you choose to make all virtual functions in a base class pure?
10. How does dynamic binding differ from static binding in C++?

12.9 REFERENCES

- "The C++ Programming Language" by Bjarne Stroustrup:
 - Author: Bjarne Stroustrup (the creator of C++)
 - Description: This is considered the definitive book for learning C++. It covers the language in-depth and provides insights into its design and evolution.
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo:
 - Authors: Stanley B. Lippman, Josée Lajoie, Barbara E. Moo
 - Description: Known for its clear and concise style, this book is suitable for beginners and serves as an excellent reference for more advanced topics.
- "Effective C++: 55 Specific Ways to Improve Your Programs and Designs" by Scott Meyers:
 - Author: Scott Meyers

- Description: This book provides practical advice and tips for writing better C++ code. It focuses on best practices and common pitfalls.
- "Accelerated C++: Practical Programming by Example" by Andrew Koenig and Barbara E. Moo:
 - Authors: Andrew Koenig, Barbara E. Moo
 - Description: Aimed at those new to programming, this book takes a practical approach to learning C++. It emphasizes writing real programs early in the learning process.
- "C++ Standard Library - A Tutorial and Reference" by Nicolai M. Josuttis:
 - Author: Nicolai M. Josuttis
 - Description: This book provides a comprehensive guide to the C++ Standard Library. It covers the containers, algorithms, and other components of the library, offering both tutorials and reference material.

UNIT 13: FRIEND FUNCTIONS FRIEND CLASS

CONTENT:

▪ *Objectives*

13.1 Introduction

13.2 Friend Function

13.3 Friend Classes

13.4 Use Cases

13.5 Summary

13.6 Keywords

13.7 Exercise

13.8 References

OBJECTIVES:

After studying this unit, you will be able to:

- Understand the need for friend functions, enabling interaction between distinct classes and accessing private data.
- Learn how to declare and use friend functions, allowing manipulation of private data from multiple classes.
- Explore the concept of friend classes, where all member functions gain access to private data, promoting flexibility.
- Recognize scenarios where friend functions and classes are beneficial, balancing access and encapsulation.

13.1 INTRODUCTION

The principles of encapsulation and data hiding suggest that functions not belonging to a class should not have access to the private or protected data of an object. The rule is clear: if you're not a member, you can't access it. However, there are instances where such strict rules can be quite inconvenient.

13.2 FRIEND FUNCTION

Consider a scenario where you need a function to work with objects from two distinct classes. Maybe the function takes objects of these two classes as arguments and manipulates their private data. In such cases, a friend function becomes invaluable. Here's a straightforward example:

```
#include <iostream>

using namespace std;

class beta;    //needed for friend func declaration

class alpha
{
private:
    int data;

public:
    alpha() : data(3) { }           //no-arg constructor

    friend int frifunc(alpha, beta); //friend function
};

class beta
{
private:
```

```
int data;

public:

beta() : data(7) { }           //no-arg constructor

friend int frifunc(alpha, beta);    //friend function

};

int frifunc(alpha a, beta b)    //function definition

{

return( a.data + b.data );

}

int main()

{

alpha aa;

beta bb;

cout << frifunc(aa, bb) << endl;    //call the function

return 0;

}
```

In this code, we have two classes named alpha and beta. The constructors in these classes set their individual data members to specific values (3 in alpha and 7 in beta).

To ensure that the frifunc() function can access both private data members, we designate it as a friend function. This is done by declaring it with the friend keyword in both classes, like so:

```
friend int frifunc(alpha, beta);
```

This declaration can be positioned anywhere within the class, and its placement in either the public or private section doesn't affect its functionality.

The function `frifunc()` takes objects from both classes as arguments, enabling it to access the private data members of both classes through these parameters. The function's task is straightforward: it adds the data items and returns the sum. The `main()` program invokes this function and displays the resulting sum.

13.3 FRIEND CLASSES

The member functions of a class can all be made friends at the same time when you make the entire class a friend.

```
#include <iostream>
```

```
using namespace std;
```

```
class alpha
```

```
{
```

```
private:
```

```
int data1;
```

```
public:
```

```
alpha() : data1(99) { }           //constructor
```

```
friend class beta;               //beta is a friend class
```

```
};
```

```
class beta
```

```
{
```

```
    //all member functions can
```

```
public:        //access private alpha data
```

```
void func1(alpha a) { cout << "\ndata1=" << a.data1; }
```

```
void func2(alpha a) { cout << "\ndata1=" << a.data1; }

};

int main()

{

alpha a;

beta b;

b.func1(a);

b.func2(a);

cout << endl;

return 0;

}
```

In the alpha class, the entirety of the beta class is granted friend status. Consequently, all member functions of beta gain access to the private data of alpha, which, in this program, is the sole data item data1.

It's essential to observe that in the friend declaration, we explicitly mention that beta is a class using the class keyword:

```
friend class beta;
```

Alternatively, we could have declared beta as a class before specifying the alpha class, similar to previous examples:

```
class beta;
```

Then, within alpha, we could refer to beta without using the class keyword:

```
friend beta;
```

13.4 USE CASES

Friend functions and classes are used when:

- External entities need access to private or protected members of a class.
- Implementing operations that are logically associated with the class but are not part of the class's interface.
- Designing utility functions that operate on the class's private members.

However, the use of friend functions and classes should be limited because they can break encapsulation and hinder the maintenance of code. It's generally recommended to provide access to class members through well-defined public interfaces whenever possible.

13.5 SUMMARY

Chapter 13 delves into the concept of friend functions and friend classes in C++. The principles of encapsulation and data hiding highlight the importance of restricting access to private or protected data. However, scenarios arise where collaboration between distinct classes is necessary, leading to the introduction of friend functions.

The code exemplifies a scenario with two classes, alpha and beta, where a function, `frifunc()`, needs access to private data members of both classes. Designating `frifunc()` as a friend function in both classes allows it to manipulate these private data members. The program showcases the simplicity of adding data items from both classes and returning the sum.

Additionally, the concept of friend classes is introduced, allowing all member functions of a class to become friends simultaneously. The alpha class designates the entire beta class as a friend, granting access to its private data. The program demonstrates how member functions of beta can seamlessly access the private data of alpha.

Three key use cases for friend functions and classes are highlighted: providing external entities access to private or protected members, implementing operations logically associated with a class, and designing utility functions that operate on private members. However, a cautionary note emphasizes limiting the use of friend functions and classes to maintain code encapsulation and readability.

13.6 KEYWORDS

Encapsulation, Data hiding, Friend functions, Friend classes, Collaboration, Private data, flexibility, Code maintenance

13.7 EXERCISE

1. What is the primary purpose of friend functions in C++?
2. How does the friend function `frifunc()` in the provided code access private data?
3. In the context of friend classes, what happens when an entire class is designated as a friend?
4. Explain the significance of the `friend` keyword in the declaration of a friend function.
5. What is the role of friend functions and classes in promoting collaboration between different classes?
6. Can you provide an example scenario where the use of friend functions or classes is justified?
7. Why is it essential to limit the use of friend functions and classes?
8. How does the placement of the friend declaration affect the functionality of a friend function?
9. What are the potential drawbacks of extensively using friend functions and classes in a codebase?
10. How does the code in the second example showcase the interaction between friend classes in C++?

13.8 REFERENCES

- "The C++ Programming Language" by Bjarne Stroustrup:
 - Author: Bjarne Stroustrup (the creator of C++)
 - Description: This is considered the definitive book for learning C++. It covers the language in-depth and provides insights into its design and evolution.
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo:
 - Authors: Stanley B. Lippman, Josée Lajoie, Barbara E. Moo
 - Description: Known for its clear and concise style, this book is suitable for beginners and serves as an excellent reference for more advanced topics.
- "Effective C++: 55 Specific Ways to Improve Your Programs and Designs" by Scott Meyers:

- Author: Scott Meyers
 - Description: This book provides practical advice and tips for writing better C++ code. It focuses on best practices and common pitfalls.
- "Accelerated C++: Practical Programming by Example" by Andrew Koenig and Barbara E. Moo:
 - Authors: Andrew Koenig, Barbara E. Moo
 - Description: Aimed at those new to programming, this book takes a practical approach to learning C++. It emphasizes writing real programs early in the learning process.
- "C++ Standard Library - A Tutorial and Reference" by Nicolai M. Josuttis:
 - Author: Nicolai M. Josuttis
 - Description: This book provides a comprehensive guide to the C++ Standard Library. It covers the containers, algorithms, and other components of the library, offering both tutorials and reference material.

UNIT 14: C-STRINGS

CONTENT:

▪ Objectives

14.1 Introduction

14.2 C-String Variables

14.3 Avoiding Buffer Overflow

14.4 Reading Embedded Blanks

14.5 Reading Multiple Lines

14.6 The Standard C++ string Class

14.7 Summary

14.8 Keywords

14.9 Exercise

14.10 References

OBJECTIVES:

After studying this unit, you will be able to:

- Grasp the significance of C-strings, exploring their historical association with C and their usage in C++.
- Learn to declare and manipulate C-string variables, considering memory allocation and the importance of null termination.
- Address the potential issues of buffer overflow by instructing input operators to restrict characters, ensuring program stability.
- Explore the advantages of the string class, understanding memory management, operator overloading, and enhanced efficiency.

14.1 INTRODUCTION

At the outset of this chapter, we highlighted the prevalence of two types of strings commonly employed in C++: C-strings and strings instantiated as objects of the string class. In this segment, we will elucidate the former, aligning with the chapter's focus as C-strings are essentially arrays of type char. These are commonly referred to as C-strings or C-style strings, owing to their historical association with the C language and their initial prominence in the early stages of C++.

Despite the ascendancy of strings generated through the string class, which will be explored in the subsequent section, C-strings remain significant for several reasons. Firstly, they find application in numerous C library functions. Secondly, their presence endures in legacy code, persisting for years to come. Lastly, for students grappling with C++, C-strings offer a more foundational and comprehensible understanding due to their inherent primitiveness.

14.2 C-STRING VARIABLES

Similar to other data types, strings can exist as variables or constants. Before delving into more intricate string operations, let's first explore these two aspects. Consider the following example that declares a singular string variable (assuming, in this context, that the term "string" denotes a C-string). The program prompts the user to input a string, captures that input in the designated string variable, and subsequently showcases the entered string.

```
#include <iostream>

using namespace std;

int main()

{

    const int MAX = 80;                //max characters in string

    char str[MAX];                    //string variable str

    cout << "Enter a string: ";

    cin >> str;                        //put string in str
```

```
//display string from str

cout << "You entered: " << str << endl;

return 0;

}
```

Each character occupies 1 byte of memory. An important aspect of C-strings is that they must terminate with a byte containing 0. This is often represented by the character constant ‘\0’, which is a character with an ASCII value of 0. This terminating zero is called the null character. When the << operator displays the string, it displays characters until it encounters the null character.

14.3 AVOIDING BUFFER OVERFLOW

What occurs when the user inputs a string surpassing the capacity of the array designated to store it? As previously noted, C++ lacks an inherent mechanism to prevent a program from inserting array elements beyond the array boundaries. Consequently, an excessively long string input could potentially lead to a system crash.

Nevertheless, it is feasible to instruct the >> operator to restrict the number of characters it assigns to an array.

```
#include <iostream>

#include <iomanip>          //for setw

using namespace std;

int main()

{

const int MAX = 20;          //max characters in string

char str[MAX];              //string variable str

cout << "\nEnter a string: ";
```

```
cin >> setw(MAX) >> str;           //put string in str,

// no more than MAX chars

cout << "You entered: " << str << endl;

return 0;

}
```

In this program, the `setw` manipulator is employed to set the upper limit on the number of characters the input buffer can handle. Although the user has the flexibility to input additional characters, the `>>` operator refrains from incorporating them into the array. In practice, the operator inserts one character less than the specified limit, ensuring sufficient space in the buffer for the concluding null character.

14.4 READING EMBEDDED BLANKS

If you tried the above program with strings that contained more than one word, you may have had an unpleasant surprise. Here's an example:

Enter a string: Law is a bottomless pit.

You entered: Law

The extraction operator `>>` treats a space as a terminating character, allowing it to read strings composed of a single word. However, anything entered after a space is disregarded. For reading text containing spaces, an alternative function, `cin.get()`, is employed. This syntax denotes the use of the `get()` member function of the stream class, with `cin` being the object. The subsequent example illustrates its usage.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
const int MAX = 80;           //max characters in string

char str[MAX];                //string variable str

cout << "\nEnter a string: ";

cin.get(str, MAX);           //put string in str

cout << "You entered: " << str << endl;

return 0;

}
```

The first argument to `cin::get()` is the array address where the string being input will be placed. The second argument specifies the maximum size of the array, thus automatically avoiding buffer overrun. Using this function, the input string is now stored in its entirety.

Enter a string: Law is a bottomless pit.

You entered: Law is a bottomless pit.

14.5 READING MULTIPLE LINES

While we may have addressed the challenge of reading strings containing spaces, the issue of handling multiline strings remains. Fortunately, the `cin::get()` function provides a solution in such cases. By introducing a third argument, this function can specify the character that signals the termination of reading. The default value for this argument is the newline character ('`\n`'), but if the function is invoked with a different character as the third argument, it overrides the default and utilizes the specified character.

In the upcoming example, we invoke the function with a dollar sign ('`$`') as the third argument:

```
#include <iostream>

using namespace std;

const int MAX = 2000;        //max characters in string
```

```
char str[MAX];           //string variable str

int main()

{

cout << "\nEnter a string:\n";

cin.get(str, MAX, '$');   //terminate with $

cout << "You entered:\n" << str << endl;

return 0;

}
```

You now have the flexibility to input as many lines as needed. The function will persist in receiving characters until you input the designated termination character (or until you surpass the array's size). It's important to note that even with the '\$' character, you still need to press Enter after inputting it.

14.6 THE STANDARD C++ STRING CLASS

In Standard C++, a novel class called "string" has been introduced, offering substantial enhancements over traditional C-strings. Notably, concerns about manually creating arrays of the correct size to accommodate string variables are alleviated with the string class, as it assumes full responsibility for memory management. Furthermore, the string class supports the use of overloaded operators, enabling concatenation of string objects using the + operator, as demonstrated by the expression:

```
s3 = s1 + s2.
```

Beyond these advantages, the string class boasts increased efficiency and safety compared to C-strings. In the majority of scenarios, this new class is the recommended and preferred approach.

You can use a constructor with no arguments, creating an empty string. You can also use a one-argument constructor, where the argument is a C-string constant; that is, characters delimited

by double quotes. As in our homemade String class, objects of class string can be assigned to one another with a simple assignment operator.

```
#include <iostream>

#include <string>

using namespace std;

int main()

{

string s1("Man");           //initialize

string s2 = "Beast";       //initialize

string s3;

s3 = s1;                   //assign

cout << "s3 = " << s3 << endl;

s3 = "Neither " + s1 + " nor ";           //concatenate

s3 += s2;                   //concatenate

cout << "s3 = " << s3 << endl;

s1.swap(s2);               //swap s1 and s2

cout << s1 << " nor " << s2 << endl;

return 0;

}
```

Here's the output of SSTRASS:

s3 = Man


```
s3 = Neither Man nor Beast
```

```
Beast nor Man
```

In the provided code snippet, the initial three lines showcase various methods of defining string objects. The first two lines involve initializing strings, while the third creates an empty string variable through a straightforward assignment using the = operator.

The string class incorporates multiple overloaded operators. Although a detailed understanding of operator overloading is deferred until the next chapter, you can readily employ these operators without delving into their internal mechanisms.

Specifically, the overloaded + operator facilitates the concatenation of one string object with another. For instance, the statement `s3 = "Neither " + s1 + " nor ";` places the string "Neither Man nor " into the variable s3.

Additionally, the += operator proves useful for appending a string to the end of an existing one. For example, the statement `s3 += s2;` adds the content of s2, which is "Beast," to the end of s3, resulting in the string "Neither Man nor Beast," assigned to s3.

This example also introduces our first string class member function: `swap()`, which exchanges the values of two string objects.

14.7 SUMMARY

Chapter 14 delves into the exploration of C-Strings, focusing on their relevance in C++ alongside the more modern string class. Initially, it highlights the primitive nature of C-strings, clarifying their role as arrays of type char and their historical connection to the C language. Despite the prevalence of the string class, C-strings remain vital for their application in C library functions, persistence in legacy code, and foundational value for students.

The chapter covers essential aspects, including the declaration and manipulation of C-string variables, emphasizing the significance of null termination for proper functioning.

Techniques to avoid buffer overflow during input operations are discussed, introducing the `setw` manipulator to limit character assignments.

Furthermore, the content addresses challenges in reading strings with spaces, introducing the `cin.get()` function as an alternative to handle multi-word strings. The approach to reading multiple lines using `cin.get()` with a specified termination character is also elucidated.

The chapter concludes by introducing the Standard C++ string class, highlighting its advantages over traditional C-strings. Memory management, operator overloading, and increased efficiency are emphasized. Practical examples demonstrate the ease of use and enhanced functionality offered by the string class, making it a recommended choice in most scenarios.

14.8 KEYWORDS

C-Strings, String Class, Buffer Overflow, Null Termination, `cin.get()`, Operator Overloading, Memory Management, Legacy Code.

14.9 EXERCISE

1. What is the historical association between C-strings and the C language?
2. How is memory allocated for C-strings, and why is null termination important?
3. How does the `setw` manipulator help prevent buffer overflow during string input?
4. What issues can arise when reading strings with spaces using the `>>` operator?
5. Explain the purpose of the `cin.get()` function and its advantages in handling multi-word strings.
6. How does the `cin.get()` function handle the termination of reading strings?
7. What are the advantages of the Standard C++ string class over traditional C-strings?
8. In what scenarios is the string class recommended, and why?
9. Describe the process of concatenating strings using the `+` and `+=` operators with the string class.
10. What is the significance of the `swap()` function in the context of the string class?

14.10 REFERENCES

- "The C++ Programming Language" by Bjarne Stroustrup:
 - Author: Bjarne Stroustrup (the creator of C++)
 - Description: This is considered the definitive book for learning C++. It covers the language in-depth and provides insights into its design and evolution.
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo:
 - Authors: Stanley B. Lippman, Josée Lajoie, Barbara E. Moo

- Description: Known for its clear and concise style, this book is suitable for beginners and serves as an excellent reference for more advanced topics.
 - "Effective C++: 55 Specific Ways to Improve Your Programs and Designs" by Scott Meyers:
 - Author: Scott Meyers
 - Description: This book provides practical advice and tips for writing better C++ code. It focuses on best practices and common pitfalls.
 - "Accelerated C++: Practical Programming by Example" by Andrew Koenig and Barbara E. Moo:
 - Authors: Andrew Koenig, Barbara E. Moo
 - Description: Aimed at those new to programming, this book takes a practical approach to learning C++. It emphasizes writing real programs early in the learning process.
 - "C++ Standard Library - A Tutorial and Reference" by Nicolai M. Josuttis:
 - Author: Nicolai M. Josuttis
 - Description: This book provides a comprehensive guide to the C++ Standard Library. It covers the containers, algorithms, and other components of the library, offering both tutorials and reference material.
-